

Hochschule München University of Applied Sciences

**Fakultät für Maschinenbau, Fahrzeugtechnik
und Flugzeugtechnik**



Projektarbeit M4000

Entwicklung und Programmierung von Modellrobotern

Ω -Tron

Verfasser :	Çurumi, Arlind
	Häußler, Christoph
	Qiu, Jiang Qi
	Güney, Ismail
Studiengang:	Maschinenbau, MBB 6
Betreuer:	Prof. Dr. T. Küpper
	Prof. Dr. J. Reichel
Abgabetermin:	13.10.2021

Inhaltsverzeichnis

1	Einleitung.....	3
1.1	Aktuelle Entwicklungstrends im Bereich Autonomes Fahren	3
1.2	Projektumfang und Kurzfassung	5
2	Projektplanung	5
2.1	Anforderungen und Zielsetzung	5
2.2	Konzeptideen.....	6
2.2.1	Fahrspurerkennung	6
2.2.2	Hindernisumfahrung (Obstacle Avoidance)	9
2.2.3	Objekterkennung zur Abfallsammlung.....	10
2.2.4	Motorensteuerung	11
2.2.5	Ortung und Navigation	12
2.2.6	Roboterarm	12
2.2.7	Induktives Laden.....	14
3	Konzeptentwicklung	14
3.1	Konzeptentwicklung für die Hindernisumfahrung	14
3.2	Computer Vision Schnittstelle	16
3.2.1	Entwicklung eines Algorithmus zur Fahrspurerkennung.....	16
3.2.2	Umsetzung einer Deep Learning Model zum Fahrspurerkennung	21
3.2.3	Objekterkennung und Schnittstelle zu Roboterarm.....	27
3.3	Modulare Umsetzung und Kommunikation im Gesamtsystem	29
3.4	Konstruktion von Fehlenden Bauteilen	30
4	Probleme bei Umsetzung und deren Behebung	32
4.1	Fahrspurerkennung	32
4.1.1	Bildverarbeitung	32
4.1.2	Deep Learning Model	32
4.2	Objekterkennung.....	33
4.3	Roboterarm	33
4.4	Hindernisumfahrung	34
4.5	Motorsteuerung	34
4.6	Ortung und Navigation	34
4.7	Induktives Laden.....	34
5	Zusammenfassung und Ausblick	35
6	Anhang.....	36
6.1	Anforderungsliste	36

6.2	Projektplan	36
6.3	Bestellliste	36
6.4	Programcode	36
6.4.1	Motorsteuerung	36
6.4.2	Hindernisumfahrung	36
6.4.3	Fahrspurerkennung	36
6.4.4	Objekterkennung.....	36
6.4.5	Roboterarm	36
6.5	Datenblätter	Fehler! Textmarke nicht definiert.
	Abbildungsverzeichnis.....	37
	Literaturverzeichnis.....	38
	Abkürzungsverzeichnis	40

1 Einleitung

1.1 Aktuelle Entwicklungstrends im Bereich Autonomes Fahren

Die Automobilindustrie befindet sich in einem massiven und komplexen, vor allem aber sehr dynamischen Wandel. Die Entwicklung neuer Fahrtechnologien, die Digitalisierung und das Aufkommen neuer Geschäftsmodelle haben diesen Veränderungsprozess bestimmt und ein einzelnes Auto zu einem Kommunikationselement gemacht, das über Internet, Sensoren und Kameras mit seiner Umwelt interagiert. Dadurch können neue Player in den Markt eintreten und die Automobilindustrie von Grund auf verändern. Der harte, globale Wettbewerb um den Markt von morgen hat bereits begonnen.

Seit Anfang des letzten Jahrzehnts haben sich Automobilhersteller und ihre Zulieferer der Umsetzung des autonomen Fahrens verschrieben. Viele Testfahrzeuge mit unterschiedlichen Assistenzsystemen wurden weltweit in eigens dafür geschaffenen Testgebieten getestet und teilweise auch im realen Straßenverkehr ordnungsgemäß gekennzeichnet. Obwohl der Begriff „Auto“ die eigenständige Fortbewegung eines Fahrzeugs beschrieben hat, haben selbstfahrende Autos den Begriff der Autonomie präzisiert und in eine neue Dimension gehoben.

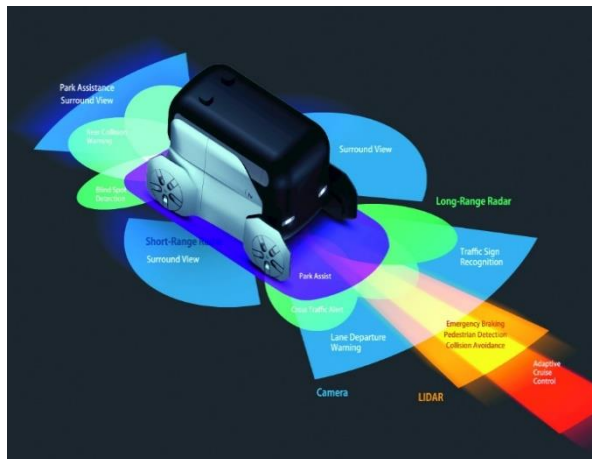


Abbildung 1: Sensortopologie für Autonome Fahrzeuge

Selbstfahrende Autos bewegen sich durch das Zusammenspiel von Mikroprozessorsystemen, Sensoren und Aktoren gezielt selbstständig. Um autonomes Fahren zu realisieren, müssen viele technische Herausforderungen bewältigt werden.

Videokameras der Umgebung, Verkehrszeichen und anderen Teilnehmern. Außerdem helfen sie dem autonomen System, die Entfernung zu Objekten richtig einzuschätzen

Radarsensoren messen den Abstand zu anderen Verkehrsteilnehmern und Objekten. Low- und High-Range-Sensoren berechnen dabei die unterschiedlichen Entfernungen. Deshalb

benötigt ein Auto mehrere davon - an verschiedenen Stellen. Lidar-Sensoren (Light Detection and Ranging) auf dem Dach tasten die vorausliegende Strecke ab. Das optische Messsystem feuert für den Menschen unsichtbare Laserstrahlen ab und berechnet den Weg der von einem Hindernis reflektierten Rückstrahlen. Lidar-Sensoren verwenden zur Messung Laserstrahlen statt Radiowellen, wie beim Radar. Ihr Vorteil: die hohe Reichweite.

Mit einem GPS-System wird das Auto genau geortet. Das System weiß immer, wo es sich gerade befindet - nicht nur auf welcher Straße, sondern auch auf welcher Spur. Das ist für abbiegende Fahrzeuge entscheidend.

Doch ein Autonomes Fahrzeug ist weitaus mehr als eine reine Fortbewegungsmittel. Sie kann auch als eine komplettes Mobilitätskonzept, aus dem wir neue Geschäftsmodelle entwickeln können,

entwickelt werden. Sie kann individuell konfiguriert werden damit Sie für alle Aufgabenstellungen innerhalb des Urbanen Umfelds einsetzbar sein kann.



Abbildung 2: Edag Citybot ©Edag

Solche vernetzten vollautonom fahrenden Roboterfahrzeuge werden zum Beispiel zur Abfallsammlung und Straßenreinigung genutzt. Das Roboterfahrzeug bekommt eine Information, wo welcher Mülleimer zu leeren ist, fährt autonom dort hin und erledigt diese Aufgabe selbstständig. Er kann die unterschiedlichen Abfallsorten durch Künstliche Intelligenz einsortieren und somit richtig entsorgen.

Weitere Konfigurationsmodelle, sind z.B. Roboterfahrzeuge, welche die Post zustellen wie der Roboter-Postbote von Amazon oder das multimodale Citybot-Konzept von EDAG .



Abbildung 3: Amazon Scout Postbote ©Amazon

Aufgrund des breiten Anwendungsbereichs und dem wachsenden Interesse an Themen wie autonomes Fahren, Bilderkennung und künstliche Intelligenz haben wir uns dazu entschieden, diese Themenbereiche auch in unserer Projektarbeit zu bearbeiten. Wir hoffen darauf, dass wir uns durch die praktischen Erfahrungen wichtige Kompetenzen für die Zukunft aneignen zu können.

1.2 Projektumfang und Kurzfassung

Ziel dieser Projektarbeit ist es, die aktuellen Entwicklungstrends aus dem Bereich autonomes Fahren, in einem Miniroboter Modellauto auf Basis von Raspberry Pi umzusetzen. Als Rechner Einheit wird ein Raspberry Pi 4B eingesetzt, welches das zentrale Element zur Steuerung der verschiedenen Module ist. Umfang der Projektarbeit war es, die elektrische Verschaltungen auszulegen, sowie Module für die Motorsteuerung, eine Fahrspurerkennung, Objekterkennung, die Steuerung des Roboterarms, Ermittlung des Ladezustands, Ansteuerung der Ultraschallsensoren und der Navigation des Roboters zu programmieren.

2 Projektplanung

2.1 Anforderungen und Zielsetzung

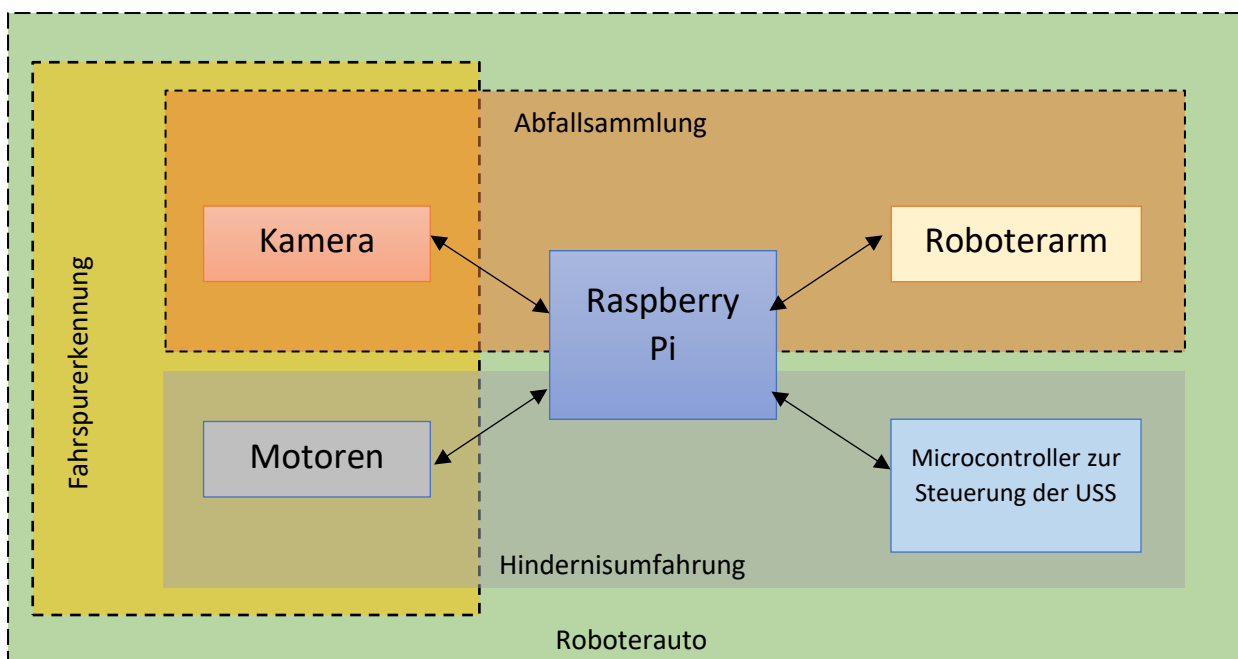
Die Anforderungen und die Zielsetzung wurde durch eine Untergliederung des Gesamtsystems ermittelt. Hierfür wurden die drei Teilsysteme Roboterauto, Objekterkennung und der Roboterarm spezifiziert. Die Schnittstellen haben sich durch diese Spezifizierung folgendermaßen ergeben:

Schnittstelle Roboterauto – Fahrspurerkennung

Schnittstelle Roboterauto – Hindernisumfahrung

Schnittstelle Roboterauto – Objekterkennung

Schnittstelle Objekterkennung – Roboterarm



Autonomes Fahren

Das Roboterauto hat als Hauptaufgabe in erster Linie die Vorwärts-, Rückwärts- und Seitenbewegung zu gewährleisten. Ziel ist es, am Ende ein voll autonom fahrendes Auto inklusive folgende Anforderungen aufzubauen:

- Hindernisumfahrung
- Fahrspurerkennung
- Objekterkennung

Abfallsammlung

Der Roboterarm ist ein auf dem Roboterauto aufgebautes System, welches vor allem für die Funktion des Greifens verantwortlich ist. Hierbei wird insbesondere durch die Schnittstelle Objekterkennung und Roboterarm Abfall aufgesammelt und entsorgt. Abgesehen vom Greifen ist es besonders wichtig, dass auch der Roboterarm automatisiert und ohne manuelle Bedienung funktioniert und den Greiferarm nutzen kann.

2.2 Konzeptideen

2.2.1 Fahrspurerkennung

Für das autonome Fahren wurde ein Algorithmus zur Fahrspurerkennung entwickelt. Dieser Algorithmus basiert auf der Hough Transformation, ein robustes Verfahren zur Erkennung von beliebigen geometrischen Figuren in einem binären Gradienten Feld. Hier handelt es sich um Geradenerkennung und berechnung der mittleren Spur zur Fahrspurverfolgung. Des weiteren wurden Datensätze gesammelt und mit ihnen trainiert, um den Roboter das Fahren einzulernen. Dafür wird ein vereinfachter CNN-Ansatz verwendet.

2.2.1.1 Bild Verarbeitung

Die Fahrspurerkennung ist einer der zentralen Bestandteile von autonomen Fahrzeugen. Es gibt viele Ansätze, um eine Fahrspurerkennung Algorithmus auszulegen. Dabei handelt es sich um das automatisierte Identifizieren der eigenen Fahrspur durch Fahrbahnmarkierungen oder andere Umgebungsmerkmale, die durch verschiedene Sensoren- und Kameras erfasst werden. Auf dem Markt gibt es einige Fahrzeuge, die über diese beiden Funktionen verfügen, nämlich den Abstandsregeltempomat (ACC) und einige Formen von Spurhaltesystemen (LKAS). Der Abstandsregler nutzt verschiedene Radarsensoren, um einen sicheren Abstand zum vorausfahrenden Fahrzeug zu erkennen und einzuhalten, während die Spurhalteassistenzsysteme mittels Kameras die Fahrspur erkennen und das Fahrzeug so lenken, dass es in der Mitte der Fahrspur bleibt.

In diesem Projekt wird ein einfacher Algorithmus zur Erkennung von parametrisierbaren geometrischen Figuren in einem binären Gradienten Bild nach einer Farbenerkennung der der Fahrspur verwendet. Dadurch wird ein einfaches und robustes Konzept zur Erkennung der Fahrspur entwickelt.

Es wurden insgesamt zwei Algorithmen entworfen, welche beide auf die Hough Transformation basieren.

Die erste Algorithmus beinhaltet die folgenden Schritte:

- (1) Bildverarbeitung zum Erkennen der Fahrspur durch Kanten Erkennung
- (2) Kanten Erkennung Algorithmus (Canny Edge Detektion)
- (3) Ausblenden von Bildbereichen, die nicht gewünscht sind (z.b. Objekte auf Seiten der Fahrspur)
- (4) Hough Transformation Algorithmus
- (5) Erzeugen von Fahrspurbegrenzungslinien zum Bilden der linken und rechten Fahrspur (Abb.4)

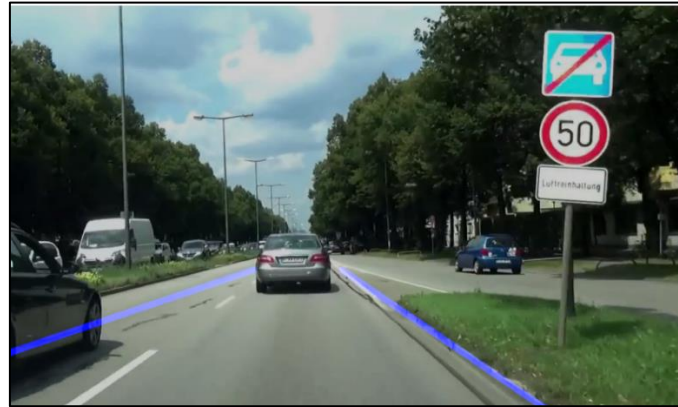


Abbildung 4: Fahrspurverfolgung durch Kantenerkennung mit Hough Transformation
Quelle: Eigene Darstellung

Der zweite Algorithmus beinhaltet die folgenden Schritten(Abb.5):

- (1) Bildverarbeitung zum Erkennen der Fahrspur durch Farbenunterschiede (Color Detection)
- (2) Kamerabildtransformation in Vogelperspektive (Birdseyeview)
- (3) Pixelaufsummierung zum Identifizieren der Fahrspur
- (4) Bestimmung der Fahrspurmittellinie

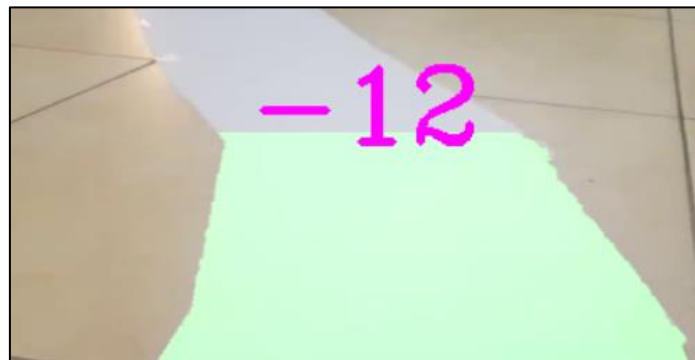


Abbildung 5: Fahrspurerkennung durch Farbenerkennung.
Quelle: Eigene Darstellung

2.2.1.2 Convolutional Neural Networks

Deep Learning ist ein Teilbereich des maschinellen Lernens, der sich mit Algorithmen befasst, die von der Struktur und Funktion des Gehirns inspiriert sind und als künstliche neuronale Netze bezeichnet werden. Im maschinellen Lernen werden Algorithmen, um eine bestimmte Aufgabe zu erledigen, ohne explizit programmiert zu werden. Anders als im maschinellen Lernen, sind Deep Learning Modelle in der Lage von sich aus zu lernen. Das passiert, in dem die Systeme das Erlernte immer wieder mit neuen Inhalten verknüpfen und dadurch erneut lernen. Bei Lernvorgang von neuronalen Netzen wird das Analysieren der neuen Daten der Maschine überlassen.

Deep-Learning Algorithmen sind Tiefe neuronale Netze und bestehen aus vielen Schichten aus linearen und nicht linearen Datenverarbeitungseinheiten.

In diesem Projekt wird auch eine Deep Learning Model angewendet, um den Modellroboter das Fahren einzulernen. Das verwendete Modell ist Convolutional Neural Network.

Ein CNN (Convolutional Neural Network) ist eine besondere Form der künstlichen Neuronalen Netzwerks die in der Bildverarbeitung und -erkennung verwendet wird. Sie ist speziell für die Verarbeitung von Pixel-Daten konzipiert.

Bei CNN Modellen werden die Eingabebilder verarbeitet und auf Bestimmten Kategorien klassifiziert. Diese Eingabebilder werden vom Computer als eine Matrix von Pixeln interpretiert. Die Größe dieser Matrix ist von Bildauflösung Abhängig.

Jedes Eingabebild, wie in Abb.6 zu sehen ist, läuft eine Reihe von verschiedenen Faltungsschichten (Eng. Layer) durch.

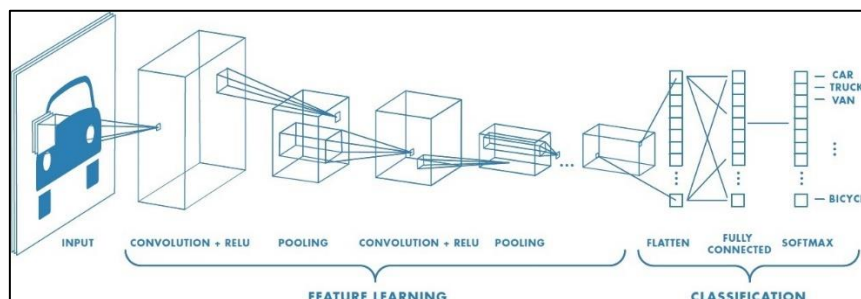


Abbildung 6: CNN Schichten

Die „Convolutional Layer“ ist die erste Schicht zur Extraktion von Merkmalen aus einem Eingangsbild. Diese Layer-Schicht bewahrt die Beziehung zwischen Pixeln, indem sie Bildmerkmale anhand kleiner Quadrate von Eingabedaten lernt. Es handelt sich um eine mathematische Operation, die zwei Eingaben wie eine Bildmatrix und einen Filter oder Kernel benötigt.

Die Schichten einer CNN sind eine Kombination aus Faltungsschichten, Pooling-Schichten, Normalisierungsschichten und vollständig verbundenen Schichten. CNNs verwenden mehrere Faltungsschichten, um Eingabevolumen auf einem höheren Grad an Abstraktion zu filtern. CNNs verbessern ihre Erkennungsfähigkeit für ungewöhnlich platzierte Objekte, indem sie Pooling-Schichten für eine begrenzte Translations- und Rotationsinvarianz verwenden. Pooling ermöglicht auch die Verwendung von mehr Faltungsschichten, indem es den Speicherverbrauch reduziert. Normalisierungsschichten werden verwendet, um lokale Eingangsbereiche zu normalisieren, indem alle Eingaben in einer Schicht auf einen Mittelwert von Null und eine Varianz von Eins gebracht werden.

Andere Techniken zur Anpassung wie z.B. „Batch“-Normalisierung bei der die Aktivierungen für den gesamten Faltungssatz normalisiert werden, oder die Dropout-Technik, bei der zufällig ausgewählte Neuronen während des Trainingsprozesses ignoriert werden, können ebenfalls verwendet werden.

Im Kapitel [3.4.2.2 Training der Daten](#) wird nochmal an das Training mit CNN angegangen und die einzelnen inneren Prozesse einer CNN-Modell nochmal mit Beispielen aus der trainierten Fahrspurerkennungsmodell der Omega-Tron erläutert.

2.2.2 Hindernisumfahrung (Obstacle Avoidance)

Wie im Abschnitt 2.2.1.1 erwähnt, verwendet man verschiedene Radarsensoren und Kameras beim Autonomen Fahren verwendet, um die Fahrspurerkennung zu ermöglichen. Um ein Fahrspurhaltesystem zu entwickeln, braucht man verschiedene Kameras und einen Algorithmus, das fähig ist Daten zuverlässig und in Echtzeit zu verarbeiten. Für den Abstandsregeltempomat (ACC) braucht man keine Kameras, jedoch wird hier auch ein Algorithmus benötigt. Dies wird benötigt um verschiedene Sensoren wie Radar-, Lidar- und Ultraschallsensoren zu steuern. Die verschiedenen Sensoren dienen dazu die Position und Geschwindigkeit des vorausfahrenden Fahrzeugs zu ermitteln.

In diesem Projekt wurde ein einfacher Algorithmus zur Erkennung von Objekten, die relativ nah zum Omega-Tron sind, verwendet. Dafür wurden keine hochwertigen Sensoren wie Radaren oder Lidar verwendet, sondern nur Ultraschallsensoren. Davon wurden 5 Stück um das Auto montiert und um den Robotermodell so montiert, dass man eine volle Abdeckung des Fahrzeugs hat.

Für das Umsetzen des Konzeptes wurden 2 Konzepte entworfen:

- (1) Steuern aller 5 Sensoren über Raspberry Pi
- (2) Steuern aller Sensoren von Arduino Nano und das Herstellen einer Master-Slave Kommunikation zwischen Raspberry Pi 4 und Arduino Nano.

Für das erste Konzept wird eine einfache Steuersoftware in Python benötigt, sowie ein Spannungswandler. Da Raspberry Pi mit 3.3V Logik arbeitet und die Ultraschallsensoren aber eine 5V Logik haben, wird eine entsprechenden Anpassung des Spannungsniveaus benötigt.

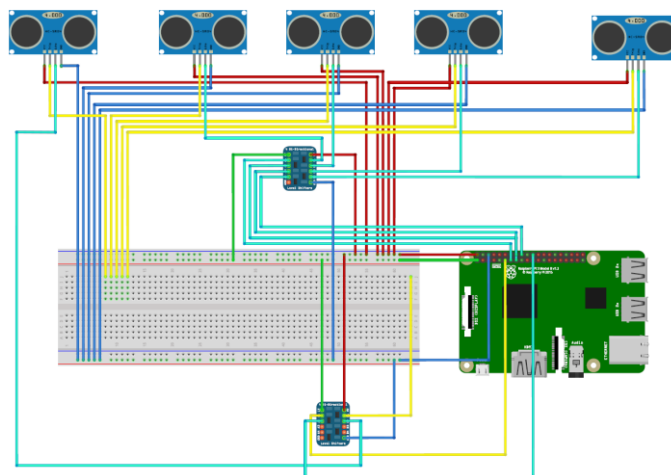


Abbildung 7: Verbindung der Raspberry Pi mit 5 USS

Beim zweiten Konzept werden die Ultraschallsensoren von einem Microcontroller gesteuert. Da der Arduino Nano eine 5V Logik hat, wird hier kein Spannungswandler benötigt. In Abb.8 sieht man die serielle Verschaltung des Raspberry Pi mit einem Arduino Nano über ein USB-Kabel.

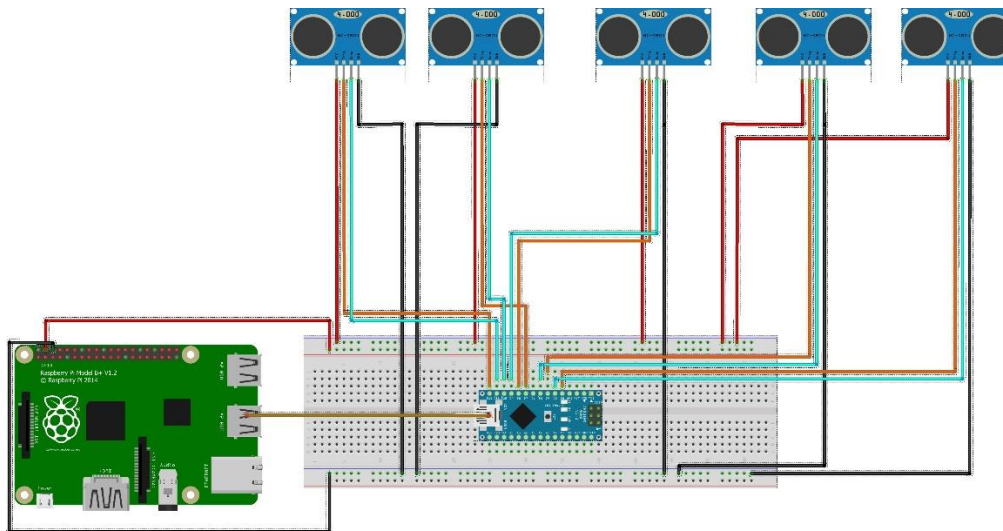


Abbildung 8: verbindung der Raspberry Pi mit Raspberry Pi Pico (Hier Arduino Nano)

2.2.3 Objekterkennung zur Abfallsammlung

Immer mehr übernehmen Computer in den verschiedensten Gebieten menschliche Aufgaben. Ein großer Bereich ist das maschinelle Lernen von Bilddaten. Egal ob das Erkennen von Fehlern bei der Produktion von SMD Platinen, die Führung eines Fahrzeugs auf der Fahrspur oder ganz groß wie in China: Das Identifizieren von Personen in der Öffentlichkeit, das automatische Erkennen von Objekten findet heute schon ganz große und wichtige Anwendung.

Ganz bekannt und einfach zu verstehen ist Image Classification. Dabei wird dem System ein Bild vorgelegt und dieses soll erkennen, um was es sich auf dem Bild handelt. Objekt Detection geht da noch ein Schritt weiter: Dabei werden um einzelne Objekte sogenannte „Bounding Boxes“ gelegt und einzeln identifiziert.

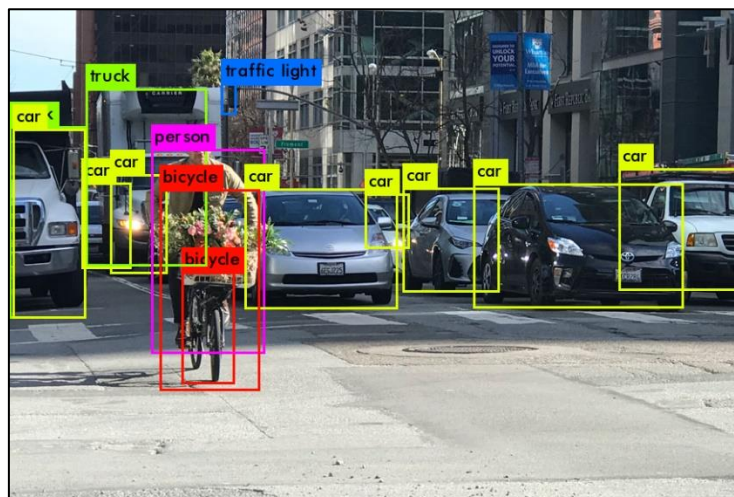


Abbildung 9: Objekt- und Positionserkennung von Fahrzeugen im Straßenverkehr

YOLO ist eine Möglichkeit, um Objekte zu erkennen. 2016 stellten Redmon, Divvala, Girshick und Farhadi erstmal ihre einstufige Architektur namens „You Only Look Once“ zur Objekterkennung in Echtzeit vor. Durch das Zusammenführen der Lokalisierung und Erkennung mehrerer Objekte innerhalb einer Ebene der Architektur, konnten so schnelle Inferenzzeiten erzeugt werden.

Mittlerweile gibt es verschiedenste Versionen von YOLO. Für unsere Anwendung nutzen wir YOLOv3, welches wie gefolgt funktioniert:

1. Das Bild wird in die Pixelgröße 416x416 Pixeln skaliert und ins Netz eingespeist
2. Dabei durchquert das Bild verschiedenen Schichten von Neuronen
3. Convolutional Network: Neuronen mit einem Filter wie z.B. 3x3 Zelle fährt über das ganze Bild um Merkmale zu erkennen
4. Endauflösung in je 13x13, 26x26 und 52x52 Zellen mit Informationen über das Bild
5. Pro Zelle werden 3 Bounding Boxes gezogen mit je 85 (Geometrie 4, Objekthaftigkeit 1, 80 Klassen) Werten
6. Über 10 000 Bounding Boxes werden durch Non-Maximum Suppression auf wenige reduziert
7. Bounding Boxes mit den schlechtesten Bewertungen werden rausgeschmissen
8. Finale Vorhersage mit je einer Bounding Box und Label

2.2.4 Motorensteuerung

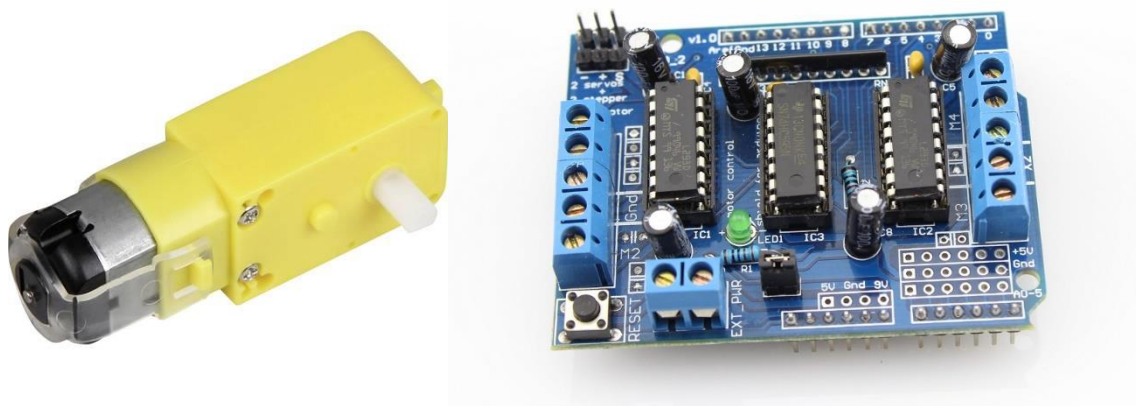


Abbildung 10: Links Getriebe Motor, Rechts Motorshield

Zur Steuerung der vier JOY-IT Getriebemotoren haben wir ein Arduino Motor Shield L293D gewählt welches wir mit Hilfe der Python Klasse AMSpi (<https://github.com/lipoja/AMSpi#readme>) steuern. Die Klasse verfügt über Befehle zum einzelne Motoren mit Drehrichtung und Geschwindigkeit anzusteuern. Mit diesen Befehlen haben wir eine Motorsteuerung programmiert, mit der man einfache Richtungsbefehle geben kann oder auch eine Geschwindigkeit und einen Lenkwinkel stufenlos vorgeben kann. Mit diesen Befehlen und der Hilfe der pygame Bibliothek, haben wir dann noch ein Programm geschrieben, um den Roboter mit den aus Computerspielen intuitiven Tastatureingaben zu steuern. Die Funktion für Lenkwinkel und Geschwindigkeit wurde noch bei der Fahrspurerkennung genutzt.

2.2.5 Ortung und Navigation

Zur Navigation des Roboters wird ROS(Robot Operating System) verwendet durch ROS erhält man Zugriff auf verschiedenste Module zur Robotersteuerung. Verwendet wird das Odometry Package zur Positionsbestimmung des Roboters. Das Modul vereint Eine Integration der Steuerungsdaten sowie Beschleunigungsdaten der IMU(InternalMeasurementUnit). Mit Hilfe der Ultraschallsensordaten mappt das Modul TIAGo den Raum und navigiert autonom durch die selbst erstellte Karte.

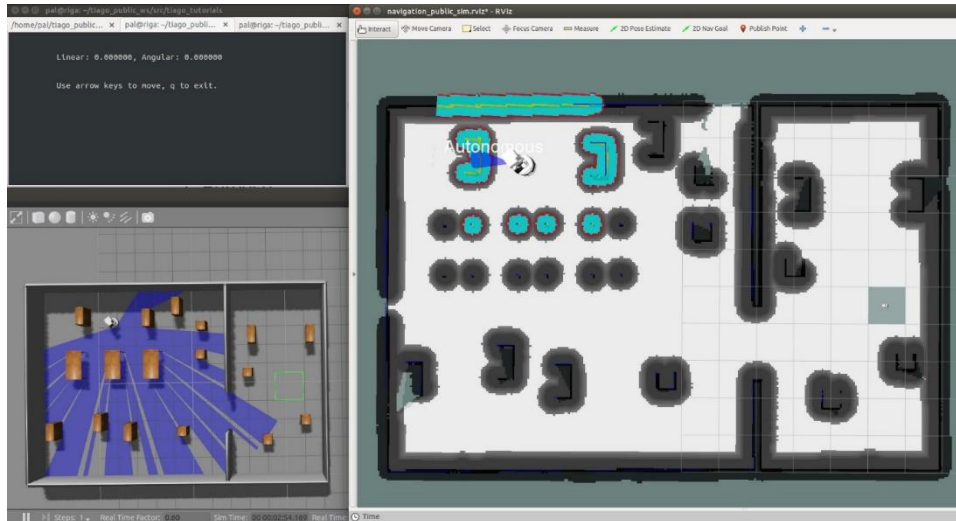


Abbildung 11: Beispielbild TIAGo Map

2.2.6 Roboterarm

Beim Roboterarm ist die Entscheidung sehr schnell auf einen Greiferarm mit vier Freiheitsgraden und Servomotoren als Antrieb für diese gefallen. Hierbei musste man auf die Art des Servomotors Acht geben, da nicht alle Gelenke des Roboterarms die gleiche Versorgung beziehungsweise manche Gelenke mehr Power als andere benötigen. Hierbei handelt es sich um die Servomotoren MG905 und MG996R. Für die Schnittstelle zum Raspberry Pi wurde der Servotreiber PCA9685 mit 16 Kanälen als beste Lösung auserkoren. Die Verbindung zwischen dem Raspberry Pi und dem Servotreiber wurde über Jumper Kabel realisiert. Die Servomotoren wurden schließlich mit dem Servotreiber verbunden und die Kanäle definiert.

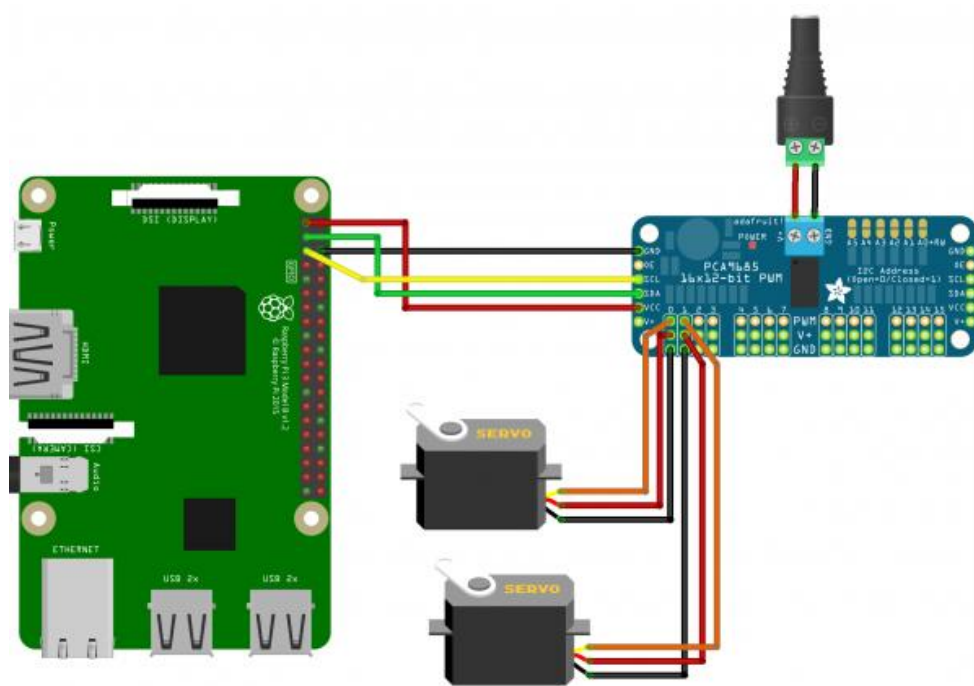


Abbildung 12: Schaltschema Servomotoren

Des Weiteren war bei der Realisierung wichtig einzuplanen, wo und wie der Roboterarm und der Servotreiber auf dem Auto eingebaut wird. Letztendlich wurde dazu entschieden, den Greifer und Treiber auf die höchste Ebene des Roboters zu montieren.

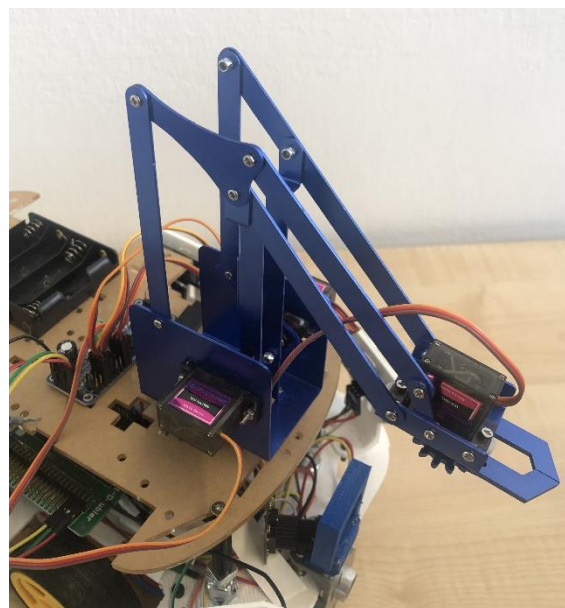


Abbildung 13: Roboterarm

2.2.7 Induktives Laden

Das induktive Laden wird durch eine Ladespule am Boden des Roboters realisiert. Der Roboter navigiert mit Hilfe der Ordometriedaten/mit ROS grob zur bekannten Position der Ladestation, wenn die mit einem GPIO verbundene Ladeplatine einen niedrigen Akkustand meldet. Die exakte Positionierung findet über die Kamera, die einen QR-Code über der Station erkennt, statt.

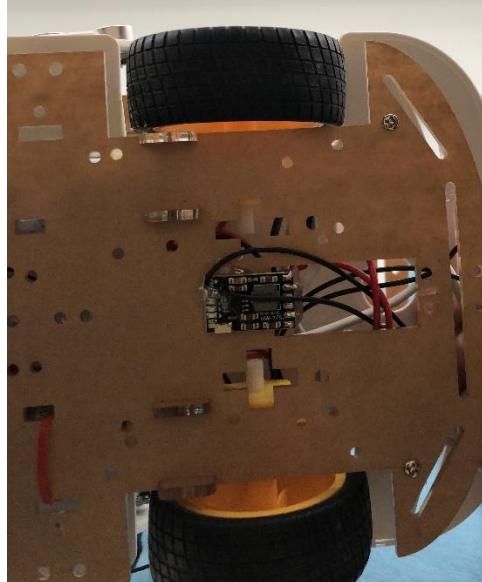


Abbildung 14: Ladeplatine mit Verkabelung Ladezustandsmessung

3 Konzeptentwicklung

3.1 Konzeptentwicklung für die Hindernisumfahrung

Beim Entwicklung des Hindernisumfahrung Konzeptes wurde beide Konzept Ideen aus der Konzeptphase ausprobiert und getestet.

Beim Ersten Konzept ging es um eine Steuerung aller 5 Sensoren direkt von Raspberry Pi. Beim Umsetzen und Testen des Steuerungssoftware wurde festgestellt, dass das Senden und Empfangen der Signale vom Raspberry Pi an die Sensoren und Rückwärts viel zu langsam ist. Außerdem gab es Komplikationen bei der Verschaltung der einzelnen Pins von der Ultraschallsensoren zu Raspberry.

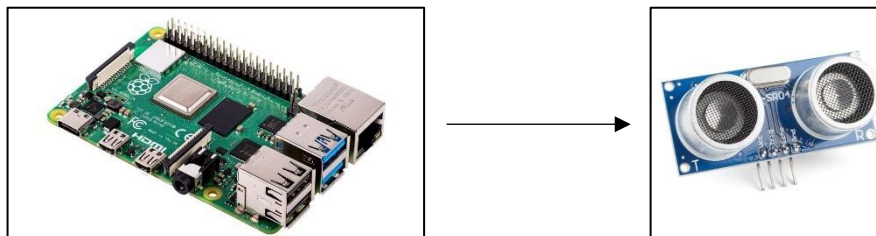


Abbildung 15: Raspberry Pi 4 Kommunikation mit USS

Um die Oberen 2 Probleme zu lösen, wurde das zweite Konzept ausprobiert und getestet, nämlich das Steuern der Ultraschallsensoren von einem Microcontroller und das Herstellen einer Master-Slave Kommunikation zwischen den Microcontroller und Raspberry Pi .

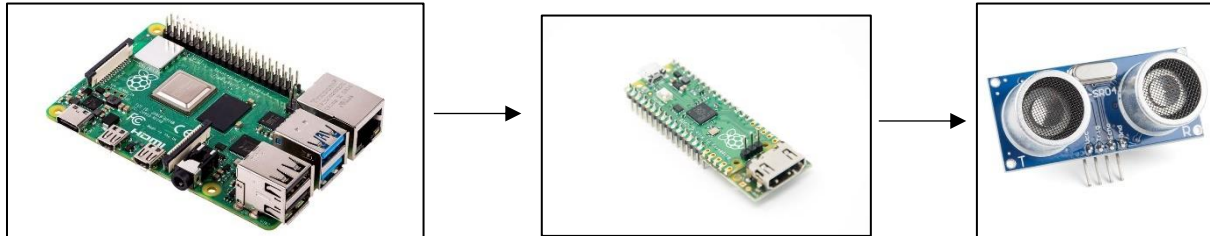


Abbildung 16: Raspberry Pi I2C mit Raspberry Pico

Erstmals wurde versucht eine Serielle Kommunikation zwischen den Raspberry Pi und Raspberry Pi Pico herzustellen. Es wurde versucht eine I2C Serielle Kommunikation zwischen den beiden herzustellen. Nach mehreren Testcodes und versuchen hat sich herausgestellt, dass das Herstellen der Kommunikation aufgrund von Bibliotheken Problemen zwischen Python 3 (In Raspberry Pi) und Micro Python (in Raspberry Pico) nicht möglich war. Da man beim Raspberry Pi Pico auch noch in C/C++ programmieren kann wurde auch noch ein letztes Versuch durchgeführt. Hierfür wurde die Programmierungsumgebung von Arduino IDE verwendet. Bei der Steuerung der Sensoren, war das Senden und Empfangen der Daten deutlich schneller, jedoch hat die Kommunikation zwischen Pi und Pico wiederum nicht zuverlässig funktioniert.

Die andere Möglichkeit war das Verwenden einer Arduino Nano zur Steuerung der Sensoren. Die Verbindung der Arduino zu Raspberry wurde mittels USB-Datenkabel ganz simpel und problemlos hergestellt.

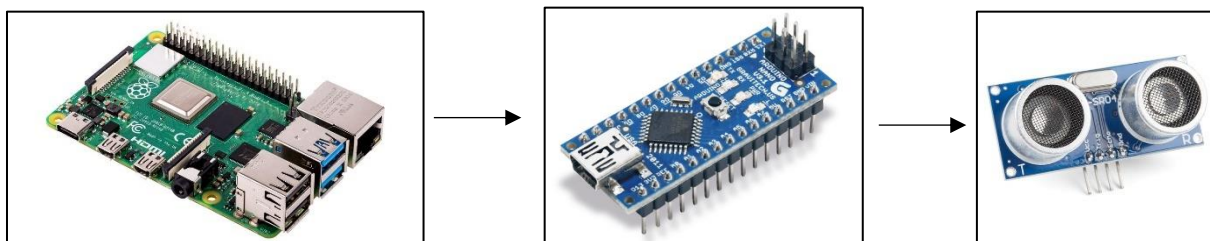


Abbildung 17: Raspberry Pi Serielle Kommunikation mit Arduino Nano

Die Kommunikation zwischen den Raspberry und dem Arduino lief zuverlässig und reibungslos. Die Serielle Kommunikation wurde auf einen Baudrate von 9600 eingestellt. Über die Python `serial()` Bibliothek werden die von Arduino Nano Print-Befehle ausgelesen.

Die Werte die Raspberry von Arduino liest werden in die Variable `Botschaft` gespeichert. Um zu unterscheiden welche Sensor welcher Wert geschickt hat, wurde eine ID Wert an die Sensoren zugewiesen. Außerdem wurde in Arduino die Reichweite der Sensoren begrenzt, die können alle jeweils bis 500 cm Reichweite auslesen. Die ID-s wurden wie Folgend definiert:

Sensor Vorne	→	0	<	Gelesene Wert	<	500
Sensor Rechts Vorne	→	500	<	Gelesene Wert + 500	<	1000
Sensor Links Vorne	→	1000	<	Gelesene Wert + 1000	<	1500
Sensor Rechts	→	1500	<	Gelesene Wert + 1500	<	2000
Sensor Links	→	2000	<	Gelesene Wert + 2000	<	2500

Im Raspberry Pi wurde dann eine Code, die je nach gelesene ID in die Zugehörige Variable die Werte speichert. Um dann das Fahrzeug zu steuern, wurden If-Anweisungen geschrieben, die beim Vergleichen von Entfernungen in eine Definierte Reichweite den Modellroboter lenken.

3.2 Computer Vision Schnittstelle

3.2.1 Entwicklung eines Algorithmus zur Fahrspurerkennung

Ein Fahrspurassistentensystem besteht aus zwei Grundbausteinen, nämlich Fahrspurerkennung (Wahrnehmung der Umgebung) und der Lenkung (Bewegungsplanung).

Die Aufgabe der Fahrspurerkennung besteht darin, ein Video der Straße in die Koordinaten der erkannten Fahrspurlinien umzuwandeln. Um dies zu erreichen, wird des Weiteren das Open-Source Package OpenCV benutzt.

Bevor wir jedoch Fahrspurlinien in einem Video erkennen können, müssen wir in der Lage sein, Fahrspurlinien in einem einzelnen Bild zu erkennen. Sobald wir das können, ist die Erkennung von Fahrspurlinien in einem Video eine einfache Wiederholung der gleichen Schritte für alle Bilder in einem Video.

Um diese Spurhalteassistentensystem zu entwickeln, wurden die folgenden 5 Schritten benötigt:

- i. Bild Verarbeitung zum Erkennen der Fahrspur (Color Detection)

Als erster Schritt muss eine Bildverarbeitungsmethode durchgeführt werden, um die Fahrspur zu erkennen. Dies kann man durch Erkennung von Farbenunterschieden ermitteln.

Da die von der Kamera aufgenommenen Frames einen RGB-Farbenspektrum haben, wird dies in den HSV-Farbenspektrum umgewandelt. Die von OpenCV erfassten Bilder und Videos sind in der Regel im BGR-Format (8-Bit, Ganzzahl ohne Vorzeichen). Anders gesagt können die Erfassten Bilder als 3 Matrizen betrachtet werden: BLAU, ROT und GRÜN mit ganzzahligen Werten zwischen 0 und 255. Bei einem Bild wird jeder einzelne Pixel dann verschiedene Kontrasten von diese 3 Farben haben.

Der Grundgedanke dahinter ist, dass in einem RGB-Bild verschiedene Farben dazu führen, dass die verschiedenen Farben durch Beleuchtungen oder andere Umgebungseinflüsse nicht richtig erkannt werden. In HSV-Farbenspektrum werden alle Objekten aus der Umgebung als Schwarz-Weiß dargestellt. Der HSV-Farbraum besteht aus 3 Matrizen, Hue, Saturation und Value. In OpenCV ist der Wertebereich für die 3 Matrizen jeweils 0-179, 0-255 und 0-255. „Farbton“ (Eng. Hue) steht für die Farbe, „Sättigung“ (Saturation) für den Anteil, in dem die jeweilige Farbe mit Weiß gemischt ist, und „Wert“ (Eng. Value) für den Anteil, in dem die jeweilige Farbe mit Schwarz gemischt ist.

Das macht das Erkennen der Fahrspurlinie Pixel für Pixel viel leichter. Im Abb.5 ist eine Vergleich der Fahrspur in HSV und RGB Farbenspektrum Dargestellt

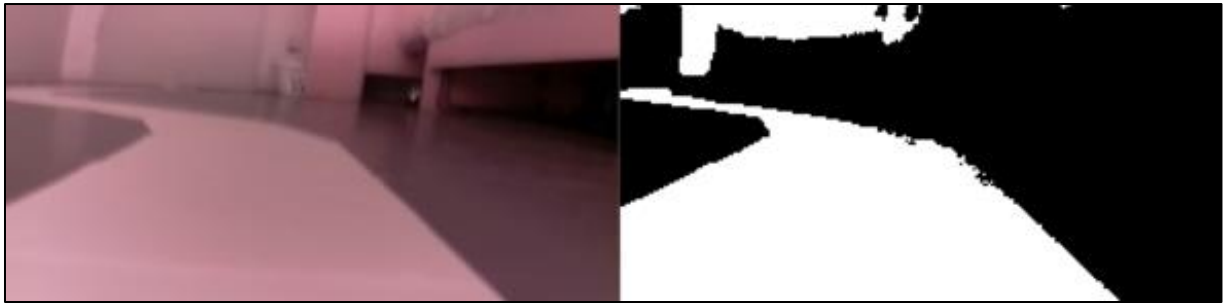


Abbildung 18: Darstellung der Fahrspur in HSV-Farben Spektrum.
Quelle: Eigene Darstellung

Zum Ermitteln der richtigen HSV Werte wird ein Skript verwendet mit dem man die Minimalen und Maximalen Werte für Hue, Saturation und Value über eine Schieberegler Fenster einstellen kann (siehe Abb.6).

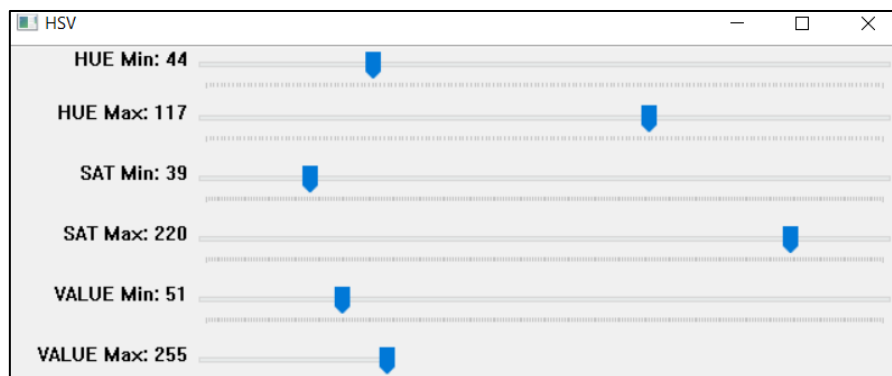


Abbildung 19: Schieberegler Fenster zum Einstellen der HSV Werte.
Quelle: Eigene Darstellung

Nach dem man ein Kompromiss zwischen alle Parametern gefunden hat, wird die Fahrspur, wie im Abb.5 Rechts, als Gradienten Bild dargestellt.

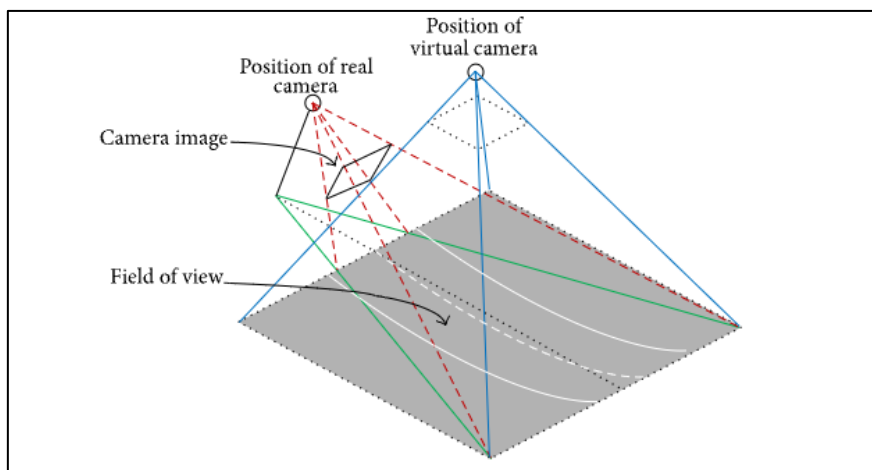


Abbildung 20: Schematische Darstellung der Bildtransformation in der Draufsicht.
Quelle: <https://www.hindawi.com/journals/js/2016/4058093/>

ii. Kamerabildtransformation in Vogelperspektive (Birdseyeview)

Durch diese Transformation des Bildes kriegt man eine Draufsicht auf die Fahrspur (siehe Abb.7). Dieser Schritt ist ausschlaggebend und dadurch wird eine präzise und effektive Erkennung der Fahrspurenkurven ermöglicht. Der Grundgedanke ist, dass mit einer Perspektive von oben der Algorithmus besser entschieden werden kann, in welche Richtung die Fahrbahn geht und wie weit die nächste Kurve ist. Um das Bild geschickt für unsere Anwendung zu verzerren, wird das OpenCV Funktion `getperspektiveTransform()` benutzt. Der Funktionen benötigen zwei Eingaben:

- Die Koordinaten der aktuellen Bild pro Frame (Abb.8 links)
- Die Koordinaten der zu transformierenden Bild pro Frame (Abb.8 rechts)

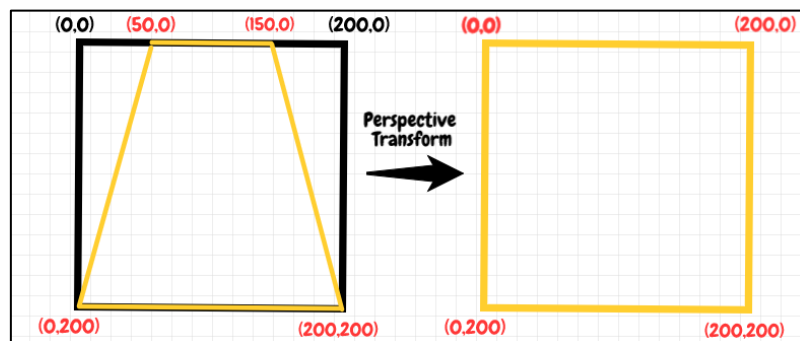


Abbildung 21: Perspective Transformation.
Quelle: medium.com

Die Koordinaten des aktuellen Bildes werden manuell eingestellt. Diese müssen so eingestellt werden, dass bei jeder Lenkung der Omega-Tron die unteren zwei Koordinaten des Vierecks immer in der Fahrspur liegen. Die Oberen zwei Koordinaten werden so eingestellt, dass die nicht zu weit, aber auch nicht zu nah mit den unteren sind. Diese zwei Punkte sind ausschlaggebend für die Lenkung des Autos. Wenn diese weit genug von den unteren Koordinaten sind, hat das Fahrzeug genug Zeit, um das nächste Lenkungsmanöver zu ermitteln. Wenn diese aber zu weit oder zu nah sind, kann das zu Verwirrungen führen, da die Fahrspurlinien dann nicht richtig erkannt werden.



Abbildung 22: Koordinaten der Aktuellen Bild (Links) und mit Transformierte Koordinaten in HSV-Farben (Rechts).
Quelle: Eigene Darstellung

Da das manuelle Einstellen der Koordinaten schwierig sein kann, wird genauso wie im vorherigen Schritt, ein Schieberegler Fenster verwendet. Damit kann man die Koordinaten in Echtzeit an das Bild einstellen. Dafür wurden die OpenCV Funktionen `cv2.createTrackbar()` und `cv2.getTrackbarPos()` verwendet. Damit man diese Punkte genauso wie in Abb.9 (Links) darstellen kann, wird die Funktion `cv2.Circle()` verwendet.

Der finale Schritt ist dann, die Koordinaten aus der *getperspektiveTransform()* an die OpenCV Funktion *warpPerspective()* zu schicken. Dadurch kriegt man einen Birdseyeview der Fahrspur wie in Abb.9 zu sehen ist.

iii. Pixeln Aufsummierung zum Identifizieren der Fahrspur

Dieser Schritt und der nachfolgende sind die wichtigsten bei der Fahrspurerkennung. Hier wird ein Algorithmus entworfen, mit dem Omega-Tron die Mitte der Fahrspur berechnen kann und somit einen Pfad verfolgen kann. Dies wird durch die sogenannte Pixel Summation Methode erreicht, die in Abb.6 dargestellt ist.

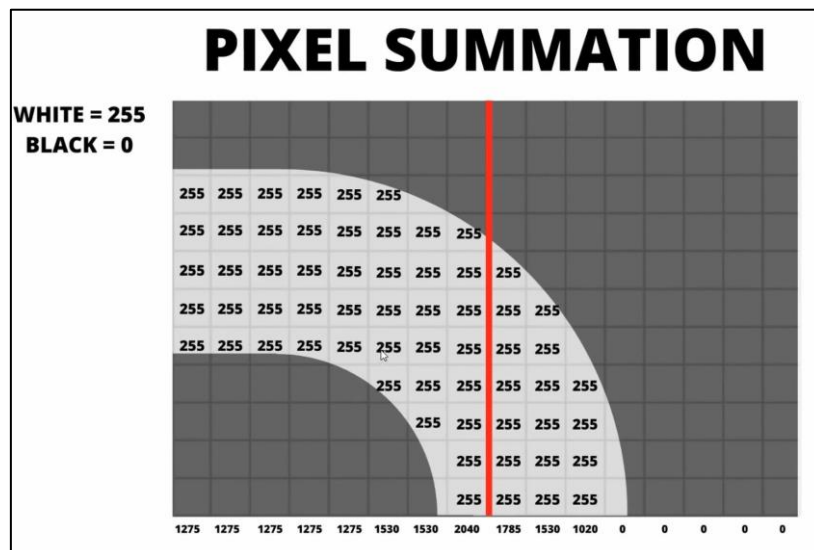


Abbildung 7: Pixeln Aufsummierung.
Quelle: @Murtaza'sWorkshop

Da das verzerrte Birdseyeview Bild nun binär ist, d. h. es hat entweder schwarze oder weiße Pixel, kann man die Pixelwerte in y-Richtung summieren, um Aussagen über die Fahrspurmittellinie zu treffen. Wie im oberen Bild dargestellt haben alle weißen Pixel einen Wert von 255 und die schwarzen einen Wert von 0.

Wenn man nun die Pixel in der ersten Spalte addiert, erhält man 1275. Diese Methode wird auf alle Spalten angewendet. In dem ursprünglichen Bild hat die Breite 480 Pixel. Daher ergeben sich 480 Werte. Nach der Summierung lässt sich feststellen, wie viele Werte über einen bestimmten Schwellenwert liegen, beispielsweise 1000 auf jeder Seite der mittleren roten Linie. Im obigen Beispiel gibt es 8 Spalten auf der linken und 3 Spalten auf der rechten Seite. Dies bedeutet, dass die Kurve nach links verläuft. Dies ist das grundlegende Konzept wie Omega-Tron die Fahrspur verfolgen kann.

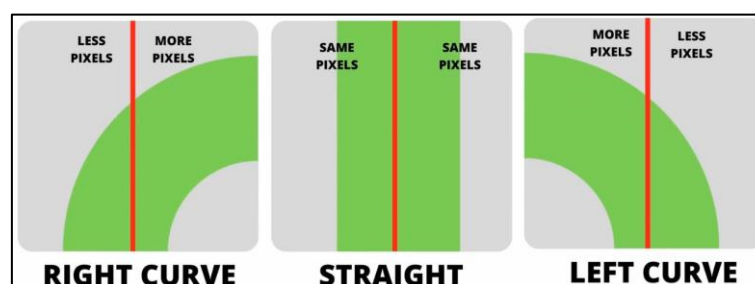


Abbildung 23: Aufsummierung der Pixeln zur Kurven Verfolgung.
Quelle: https://easychair.org/publications/preprint_download/4DRI

iv. Bestimmung der Fahrspur Mittellinie (Hough Transformation durch Pixel Summation)

Durch den Vorherigen Schritt wurde es erläutert wie das Auto durch Pixel Aufsummierung die Fahrspur verfolgen kann und somit entschieden kann, ob Sie nach links oder nach rechts lenkt.

Jedoch kann es passieren, dass es beim Fahren einer geraden Straße mehr weiße Pixel nach rechts als nach links gibt oder andersherum. In diesem Fall würde das Auto nach links bzw. nach rechts Lenken, was falsch wäre. Diese Fehler entstehen, da im vorherigen Schritt als Mitte der Fahrspur, die Mitte des Aufgenommenen Bildes verwendet wurde, was also eine fester Wert.

Dieses Problem kann leicht behoben werden, indem man die Mitte der Fahrspurlinie über den Mittelwert der weißen Pixeln in der Fahrspur ermittelt. Anhand von dieser berechneten Mittellinie werden dann genau wie im obigen Schritt die Pixeln nach links und rechts aufsummiert, um die Lenkkurve zu finden. Einfacher gesagt, die Fahrspurmitte und der benötigte Lenkwinkel wird für jede Frame berechnet, um die besten Lenkwerte zu bekommen.

Wenn die Kurve rechts ist, ist es wichtig zu wissen, wie weit rechts. Um den Wert der Krümmung zu ermitteln, sucht man die Indizes aller Spalten, deren Wert über einen gewissen Schwellenwert liegt, und bildet dann den Durchschnitt der Indizes.

Die Schwellenwert wird anhand von Versuchen festgelegt. Das ist der Mindestwert, der für jede Spalte erforderlich ist, um als Teil des Pfades und nicht als Rauschen zu gelten.

Die Berechnung des Fahrspur Mittelpunktes wurde in den Funktion `getHistogram()` mit der Variable `CurvenMittelwert` definiert.

v. Implementierung

Für das Implementieren wird ein Main Skript geschrieben. In diesem Skript wird das Kameramodul, das Fahrspurerkennungsmodul und das Motormodul aufgerufen.

Um das Fahrzeug geschickt zu lenken, wird beim Aufrufen des Fahrspurerkennungsmoduls eine Sensitivität und ein Geschwindigkeitsfaktor vordefiniert. Dies dient dazu, dass das Fahrzeug beim Fahren der Kurve nicht bei jeder Frame nach links und nach rechts lenkt, sondern nur dann, wenn ein maximaler Kurven Wert überschritten wird.

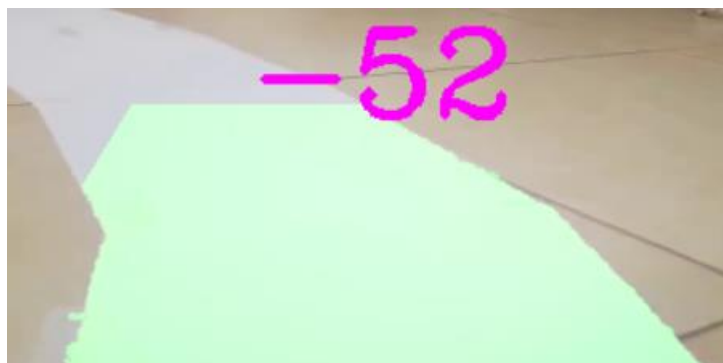


Abbildung 24: Auszug aus der Implementierung der Algorithmus

3.2.2 Umsetzung einer Deep Learning Model zum Fahrspurerkennung

3.2.2.1 Datensammlung

Der Erste Schritt, um Omega-Tron das Autonome Fahren beizubringen, ist Daten zu sammeln. Diese Daten werden dann benutzt, um ein Deep Learning Model zu trainieren. Um einen Algorithmus der Fahrspurerkennung zu lernen und somit das autonome Lenken beim Fahren, braucht man Lenkungsdaten und die entsprechenden Bilder dazu, wo der Modellroboter ein Lenkungsmanöver durchführt.

Um die Daten zu sammeln, wurde das Python Skript *LogModul* geschrieben in die Bilder und Excel Protokolldaten mit dem Bild, Name und dem entsprechenden Lenkwinkel dazu. Dieses Skript wird dann im Main Skript zum Sammeln der Daten aufgerufen, das *LogMain* Skript. Hinzu ruft diese Funktion noch die Module zur manuellen Steuerung von Omega Tron, sowie auch das Kamera Modul Skript.

Im *LogMain* werden diese dann jeweils aufgerufen und mit einem definierten Tastenbefehl wird das Aufzeichnen gestartet und gestoppt. Das Steuern von dem Roboter erfolgt entweder per Tastaturbefehl oder mit einem PS4 Controller. In Abb. 19 ist noch ein Schema des benötigten Skripts, die das *LogMain* Modul zum Datensammlung benötigt.

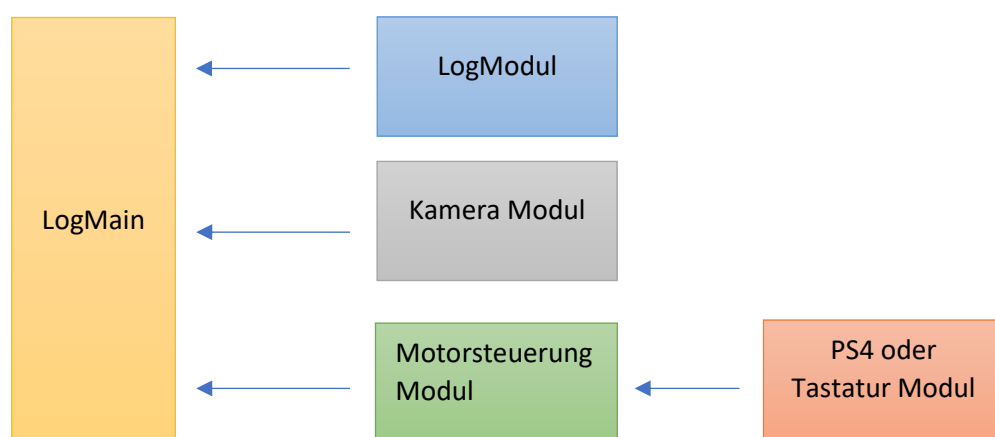


Abbildung 25: Schema der Datensammlung Programm Code

3.2.2.2 Training der Daten

Das Training der Datensätze kann man in 10 einfachen Schritten erklären.

1- Importieren der Daten Pfade

Die gesammelten Datensätze, die Bilder aus der Fahrspur mit die Entsprechenden Lenkungswerte, werden im Training Skript als Pfade geladen. Hier werden diese als Pandas Dataframes importiert. Ein Pandas Dataframe ist eine einfache Darstellung der Daten in einer Tabelle mit Reihen und Spalten. Die zwei Spalten sind die Lenkungswerte aus der Excel Datei und die Pfade der entsprechenden aufgenommenen Bilder. Der Vorteil hier ist, dass bei der Bearbeitung dieser Daten, nicht direkt auf das Dataset zugegriffen wird, sondern nur auf den Dateipfad. Zum Beispiel wenn man einige Daten aus der

Pandas Liste löschen will, werden nur die Pfade gelöscht und diese Dateien werden im Training einfach nicht mehr verwendet. An einem späteren Zeitpunkt wären diese Daten dann wieder greifbar. In Abb. 12 sind die ersten 5 Daten aus einer Gesamtanzahl von 6659 zu sehen.

```
D:\Apps\PyCharm\Projects\venv\Scripts\python.exe D:/Apps/PyCharm/Projects/AutonomesFahren/
0:2006 1:323 2:1302 3:3028
Anzahl Datein 6659
```

	Bilder	Lenkung
0	IMG0\Image_163318630696752.jpg	0.0
1	IMG0\Image_1633186307085666.jpg	0.0
2	IMG0\Image_1633186307202879.jpg	0.0
3	IMG0\Image_1633186307318438.jpg	0.0
4	IMG0\Image_1633186307429695.jpg	0.0

Abbildung 26: Auszug aus der Daten Import

2- Visualisierung und Anpassung der Daten

Bevor man die Daten trainiert ist es wichtig eine Visualisierung des Datensatzes durchzuführen. Hier wurde festgestellt, dass die Mehrheit der Bilder und Lenkungswert sind, wenn der Roboter geradeaus gefahren ist (siehe Abb.13). Im Vergleich zu den anderen Werten hat diese Kategorie viel mehr Daten und ist somit ungleichmäßig. Je ungleichmäßiger ein Datensatz ist, desto ungenauer wird das trainierte Modell am Ende sein, da dieses sehr stark auf eine Kategorie trainiert wurde, als die andere. In diesem Fall würde das Modell, dass geradeaus fährt sehr gut lernen, aber die restlichen Lenkungsmanöver wären nicht so gut trainiert.

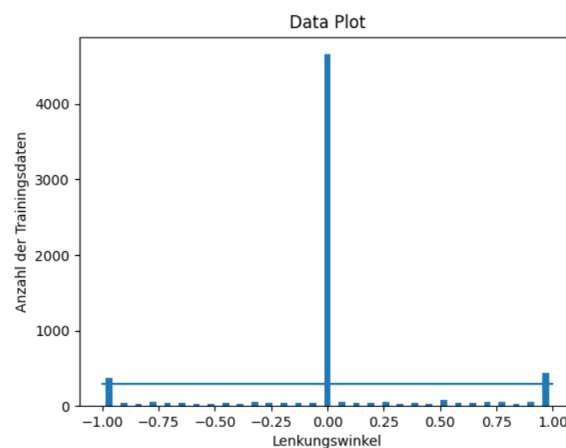


Abbildung 27: Visualisierung der Daten

Um dies zu vermeiden, werden die Daten balanciert. In diesem Fall wurden alle Kategorien mit mehr als 300 Daten geschnitten. Nach diesem Prozess wird die Datenmenge pro Lenkungsstärke wie unten aussehen.

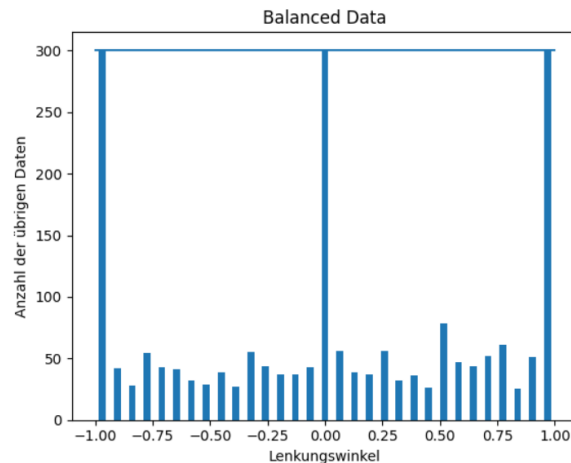


Abbildung 28: Ausgeglichene Daten

3- Laden der eigentlichen Daten zum Trainieren

In diesem Schritt werden alle Daten, die aus dem Schnitt 2 nicht geschnitten wurden, hochgeladen, um diese dann in den kommenden Schritte zu trainieren. Im Unterschied zu Schritt 1 werden hier die physikalischen Daten hochgeladen, nicht nur die Adressen und die Inhalte davon.

4- Aufteilung der Daten in Training und Validierungsdaten

Bevor man die bereits hochgeladenen Daten trainiert, ist es wichtig den Datensatz in Trainings- und Validierungsdatensätze aufzuteilen. Dies macht man, um die Effizienz des zu trainierenden Algorithmus abzuschätzen und zwar durch das Anwenden der von dem Trainingsdaten trainierten Modell auf Daten, die nicht zum Trainieren des Modells verwendet wurden, nämlich die Validierungsdaten. Um den Aufteilungsprozess (Eng. Split) leichter zu machen wird die Python Bibliothek Scikit-Learn verwendet.

Demnach werden die Daten in x und y aufgeteilt. Hier handelt es sich um einen „Unsupervised Learning“ Algorithmus, indem die Maschine den Zusammenhang der Lenkungsdaten mit den Bildern selbst entdeckt und versucht ein Modell dafür zu trainieren. Man hat Eingabevariablen (x) und Ausgabevariablen (y), mit dem der Algorithmus die Verknüpfung von der Eingabe zur Ausgabe lernt.

Die Lenkungsdaten sind also mit der entsprechenden Bildaufnahme zu diesem Lenkungsmanöver in der Excel-Datei verknüpft und der Algorithmus wird so trainiert, dass er selbst entscheidet, wann der beste Moment ist, in Bezug auf das Bild was es von der Kamera bekommt, um ein Lenkungsmanöver durchzuführen.

Der Datensatz wurde in 80% Trainingsdaten und 20% Validierungsdaten aufgeteilt. Die Daten werden zufällig aufgeteilt und es gibt keinen Auswahlprozess.

In Abb.16 ist die Anzahl der Daten, die für das Training und Validierung verwendet wurden zu sehen. Außerdem sieht man auch die Anzahl der Bilder, die während Schritt-2 geschnitten wurde.

```
Entfernte Bilder: 4568
Verbleibende Bilder: 2091
Training Bilder Gesamt: 1672
Validierung Bilder Gesamt: 419
```

Abbildung 29: Auszug aus der Programcode

5- Daten Augmentieren

Die Datenaugmentierung heißt nichts anderes, als nur Originalbilder zu modifizieren. Während der Fahrt kann sich die Fahrspurerkennung als schwierig erweisen, falls die Beleuchtung sich ändert oder eine Kurve zu eng ist und die Kamera nicht mehr die ganze Fahrspur sehen kann. Äußerem kann man in diesem Schritt den Datensatz durch das Erzeugen von diesen zusätzlichen Dateien erhöhen. Es handelt sich aber nicht nur um die Anpassung der Lichthelligkeit und Bildgröße, sondern auch um das Umdrehen des Bildes, was einen erheblichen Einfluss auf den Trainingsprozess hat.

Es werden auf Zufallsbasis 50% aller Daten augmentiert.

6- Anlegen eines Farbenfilters für den Trainingsdatensatz (Preprocessing)

Genau wie in Schritt 5 handelt es sich hier um Datenaugmentierung. Der Unterschied zu Schritt 5 ist, dass es sich hier um ein Bildverarbeitungsprozess handelt, welches auch im Fahrspurerkennung Kapitel 3.4.1 [Absatz i\)](#) verwendet wurde. Das „Preprocessing“ ist ausschlaggebend für die Farbenerkennung da hier die verschiedenen Filter angelegt werden mit dem das Fahrzeug danach die Fahrspur erkennt.

7- Erstellen eines CNN Model für das Training der Daten

Hier befindet sich man beim wichtigsten Schritt des Deep Learning Modells zur Fahrspurerkennung, das Erstellen eines CNN-Modells. Das hier verwendete Modell basiert auf das [NVIDIA Deep Learning Modell für Autonome Fahrzeuge](#). In Abb. 17 ist der Aufbau der Faltungsschichten, des Nvidia Modell zu sehen.

Die Gewichte des Netzwerks werden so trainiert, dass der mittlere quadratische Fehler zwischen dem vom Algorithmus ausgegebenen Lenkbefehl und dem durch den Augmentieren angepassten Lenkbefehl, minimiert wird.

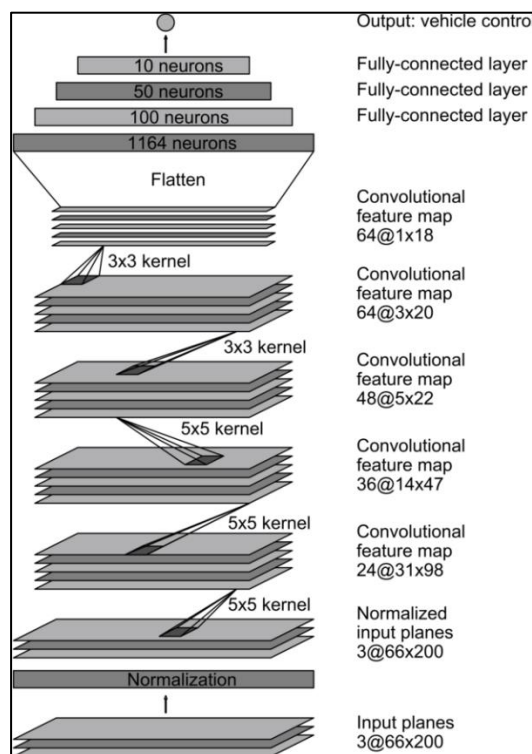


Abbildung 30: NVIDIA CNN Model
Quelle: @Nvidia.com

Das NVIDIA Model besteht aus 9 Schichten, darunter eine Normalisierungsschicht, 5 Faltungsschichten und 3 vollständig verbundene Schichten.

Die erste Schicht des Netzes führt eine Bildnormalisierung durch. Der Normalisierungsmechanismus ist fest kodiert (Eng. Hard-coded) und wird während des Lernprozesses nicht angepasst.

Die Faltungsschichten sind für die „feature extraction“ konzipiert und werden empirisch durch eine Reihe von Experimenten mit unterschiedlichen Schichtkonfigurationen ausgewählt. In den ersten drei Faltungsschichten werden geschichtete Faltungen mit einem 2×2 „Stride“ und einem 5×5 Kernel und in den letzten beiden Faltungsschichten eine nicht-„strided“ Faltung mit einer 3×3 Kernelgröße verwendet.

„Stride“ ist ein Parameter des Filters des neuronalen Netzwerks, der den Bewegungsanteil im Bild oder Video verändert. Wenn der „Stride“ eines neuronalen Netzwerks beispielsweise auf 1 eingestellt ist, bewegt sich der Filter jeweils um ein Pixel bzw. eine Einheit. Die Größe des Filters wirkt sich auf das kodierte Ausgabevolumen aus, daher wird „stride“ oft auf eine ganze Zahl und nicht auf einen Bruchteil oder eine Dezimalzahl eingestellt.

Auf diese fünf Faltungsschichten folgen dann drei vollverknüpfte Schichten, die zu einem endgültigen Ausgabesteuerungswert führen. Die vollverknüpften Schichten sind so konzipiert, dass sie als Controller für die Lenkung des Modell Roboter dienen.

8- Trainings des Models

Nach dem Erstellen des Models werden alle Prozessen aus der vorherigen Schritte in einer Funktion zum Trainieren der Daten aufgerufen. Der Funktion wurde als *dataGen()* definiert.

Zum Aufrufen der Funktion werden „Batches“ definiert. „Batches“ sind Hyperparameter, der die Anzahl der zu verarbeitenden Daten festlegt, bevor die internen Modellparameter aktualisiert werden. Ein „Batch“ lässt sich als eine „for“-Schleife vorstellen, die über eine oder mehrere Bilder und Lenkungsdaten iteriert und Vorhersagen macht. Am Ende des Stapels werden die Vorhersagen mit den erwarteten Ausgangsvariablen verglichen und ein Fehler wird berechnet. Anhand dieses Fehlers wird der Aktualisierungsalgorithmus verwendet, um das Modell zu verbessern.

Beim Trainieren wurden „Batches“ in Größe von 100 zufällig ausgewählten Daten aus dem Datensatz programmiert. Diese 100 Daten werden dann augmentiert und mit einem Farbenfilter auch Verarbeitet. Nach dem Prozess gibt die Funktion *dataGen()* einen *y* wert zurück, der von der *model.fit()* Funktion im Main Training Skript aufgerufen wird.

Die *model.fit()* Funktion ist eine Standardfunktion der Tensorflow Bibliothek zum Trainieren von Deep Learning Modelle. Hier werden die Ausgabewerte aus der *dataGen()* Funktion für die 80% Trainingsdatensätze der ganzen Daten und der 20% Validierungsdatensätze aufgerufen. Andere Inputs der *model.fit()* brauch sind „Batch-Größe“ , „Epoch“-Anzahl, und die Schritte pro „Batch“ für die Validierung und Trainingsdaten.

„Epoch“-Anzahl gibt die Anzahl der Durchläufe des gesamten Trainingsdatensatzes an, die der Algorithmus abschließen soll um das Modell fertig zu trainieren. Bei diesem Modell wurden 10 „Epoch’s“ verwendet mit jeweils 100 „Batches“. Anders gesagt, haben wir 10 Schritte zum Trainieren des Modells und jeder Schritt hat 100 zufällig ausgewählte Daten, die jeweils 100-mal trainiert werden um einen „Epoch“ abzuschließen.

In Abb18. befindet sich ein Auszug aus dem trainierten Modell, wo man die Durchläufe beim Training ansehen kann. Um den Trainingsprozess im Anschluss zu visualisieren, wird das Training durch die Funktion *model.fit()* an eine Variable gespeichert.

```
Epoch 1/10
100/100 [=====] - 88s 858ms/step - loss: 0.2852 - val_loss: 0.2518
Epoch 2/10
100/100 [=====] - 37s 371ms/step - loss: 0.2430 - val_loss: 0.2432
Epoch 3/10
100/100 [=====] - 37s 369ms/step - loss: 0.2422 - val_loss: 0.2274
Epoch 4/10
100/100 [=====] - 37s 372ms/step - loss: 0.2255 - val_loss: 0.2365
Epoch 5/10
100/100 [=====] - 37s 376ms/step - loss: 0.2084 - val_loss: 0.2220
Epoch 6/10
100/100 [=====] - 38s 382ms/step - loss: 0.2033 - val_loss: 0.1943
Epoch 7/10
100/100 [=====] - 38s 387ms/step - loss: 0.2021 - val_loss: 0.1914
Epoch 8/10
100/100 [=====] - 38s 380ms/step - loss: 0.1923 - val_loss: 0.1915
Epoch 9/10
100/100 [=====] - 38s 384ms/step - loss: 0.1985 - val_loss: 0.1838
Epoch 10/10
100/100 [=====] - 38s 379ms/step - loss: 0.1881 - val_loss: 0.1869
Model gespeichert
```

Abbildung 31: Trainieren des Models

9- Speichern des Models und Plotten der Ergebnisse

Nach dem das Training abgeschlossen ist, wird das Model gespeichert. Dies ist dann das endgültige Deep Learning Model mit dem der Omega-Tron autonom eine Fahrspur fahren kann.

Wie im oberen Abschnitt erwähnt, das Trainings Model wurde an eine Variable gespeichert. Dadurch kann man den Trainingsprozess visualisieren, um eine Aussage über die Effizienz des Modells zu treffen. Wie man in Abb.19 Erkennen kann, nimmt der Trainingsverlust mit zunehmender Anzahl von „Epoch's“, d.h. mit zunehmender Anzahl von Trainingsdaten weiter ab.

Die Grafik der Validierungsverluste nimmt bis zu einem Punkt ab und beginnt dann wieder zu steigen und nach einer Weile sinkt die wieder ab. Der Wendepunkt im Validierungsverlust kann der Punkt sein, an dem das Training abgebrochen wird, da die Erfahrung nach diesem Punkt, die Dynamik der Überanpassung (Eng. „Overfitting“) zeigt.

Alles in allem kann man herauslassen, dass das trainierte Modell ein gutes Modell ist. Diese Stellungnahme kann man mit den folgenden zwei Gründen belegen:

- Die Kurve der Trainingsverluste sinkt bis zu einem Punkt der Stabilität ab
- Die Kurve der Validierungsverluste sinkt bis zu einem Punkt der Stabilität und hat einen kleinen Abstand zu den Trainingsverlusten, was bedeutet, dass das Modell nicht „Overfitted“ wurde.

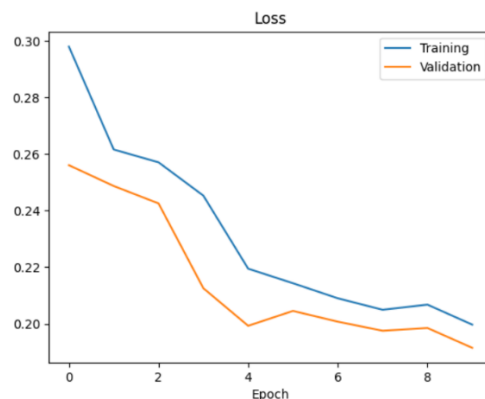


Abbildung 32: Ergebnisse aus der Training des Models

3.2.2.3 Umsetzung der CNN Model

Beim Umsetzen des CNN-Models wurde das Main Skript *Omega_Trone_CNN_Main.py* geschrieben. In diesem Skript wird das trainierte Model zusammen mit Kamera Modul und den Motormodul aufgerufen. Hier wird wiederum eine variable für die Sensitivität bei der Lenkung der Kurven sowohl als auch eine Maximale Geschwindigkeit, sodass die kurven Optimal aufgenommen werden können. Im Skript wird noch eine kleine Bildverarbeitungsfunktion geschrieben in dem alle von der Kamera Aufgenommenen Bilder pro Sekunde Augmentiert werden. Diese Augmentierte bilder werden an das Modell gesendet, die danach an den Motormodul befehle über die Lenkungsstärke übergibt. Diese Lenkungsstärke wird dann mit die definierten variablen für die Sensitivität und Geschwindigkeit Multipliziert.

3.2.3 Objekterkennung und Schnittstelle zu Roboterarm

3.2.3.1 Daten Sammlung und Training mit Yolo

Zur Bilderkennung nutzen wir ebenso die Raspberry Kamera, welche in Echtzeit Müll erkennen kann. Wie bereits in Kapitel 2.2.3. beschrieben, nutzen wir für die Erkennung YOLOv3. Doch bevor es zur Modellbildung kommt, werden zuvor Daten benötigt:

1. Rohdaten sammeln: Damit später genau die Gegenstände erkannt werden, die zuvor festgelegt worden sind, müssen entsprechende Bilder ins System eingespeist werden. Dabei nutzen wir öffentliche Bilder im Internet, als auch selbst geschossene:



Abbildung 33: Öffentliches Bild (links) und selbst geschossenes Bild (rechts)

2. Daten aufbereiten: Nach etwa 1000 Bilder mit 4 verschiedenen Klassen (Bio, Papier, Schachtel, Dose) werden nun die Daten für das Modell vorbereitet. Mithilfe der Seite roboflow.ai können folgende Schritte durchgeführt werden:

- a. Bild Labelling: Zunächst werden alle Bilder beschriftet. Dabei legt man die Bounding Box um das Objekt und versehen diesen noch mit der entsprechenden Bezeichnung.
- b. Preprocessing: Hier wird das Bild auf 416x416 Pixel skaliert, was die Standardgröße bei YOLO ist. Dadurch kann Rechenzeit gespart werden.
- c. Argumentation: Um den Trainingseffekt zu verbessern vermehren wir die Daten, indem wir die Bilder jeweils spiegeln und rotieren. Unser Datenpaket wurde somit verdreifacht.



Abbildung 34: Dose mit Bounding Box

3. Daten ins Modell einspeisen: Dem vortrainierten Modell, welches ausführlich in Kapitel 2.2.3. beschrieben wurde, füttern wir mit unseren Daten ein. Der gesamte Ablauf findet über Google Colab statt und wird im Kapitel 6.4.5. beschrieben.
4. Training: Nachdem das Modell konfiguriert wurde, startet das Training. Je mehr Zyklen durchlaufen werden, desto mehr passt sich das Modell den Daten an:

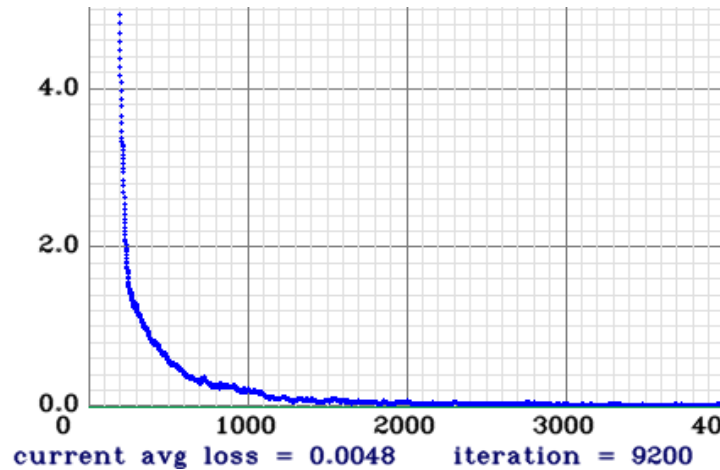


Abbildung 35: x = Anzahl Zyklen, y = Prozentualer Fehler

3.2.3.2 Schnittstelle Yolo zu Roboterarm

Bei der Objekterkennung wird über die Yolo Bibliotheken der Programmcode *Obj_Detection.py* geschrieben, indem das trainierte Modell und die Klassennamendatei alle Abfallkategorien aufgerufen werden. In dem Code wird eine Funktion definiert als *Objekt_Erkennung()*, indem beim Erkennen einer Abfallkategorie eine „Bounding Box“ um das Objekt gelegt wird. Diese „Bounding Box“ ist ein virtuelles Rechteck, das als Bezugspunkt für die Objekterkennung dient und einen Rahmen für dieses Objekt bildet. Die wird über die OpenCV Bibliothek berechnet, sodass nur das erkannte Objekt umrandet wird. Über diesem Rahmen wird außerdem der Kategoriennamen und der Wahrscheinlichkeitswert, dass das Objekt zu der Kategorie gehört. Die Kategoriennamen sind in der Klassennamen Datei wie folgt gelistet:

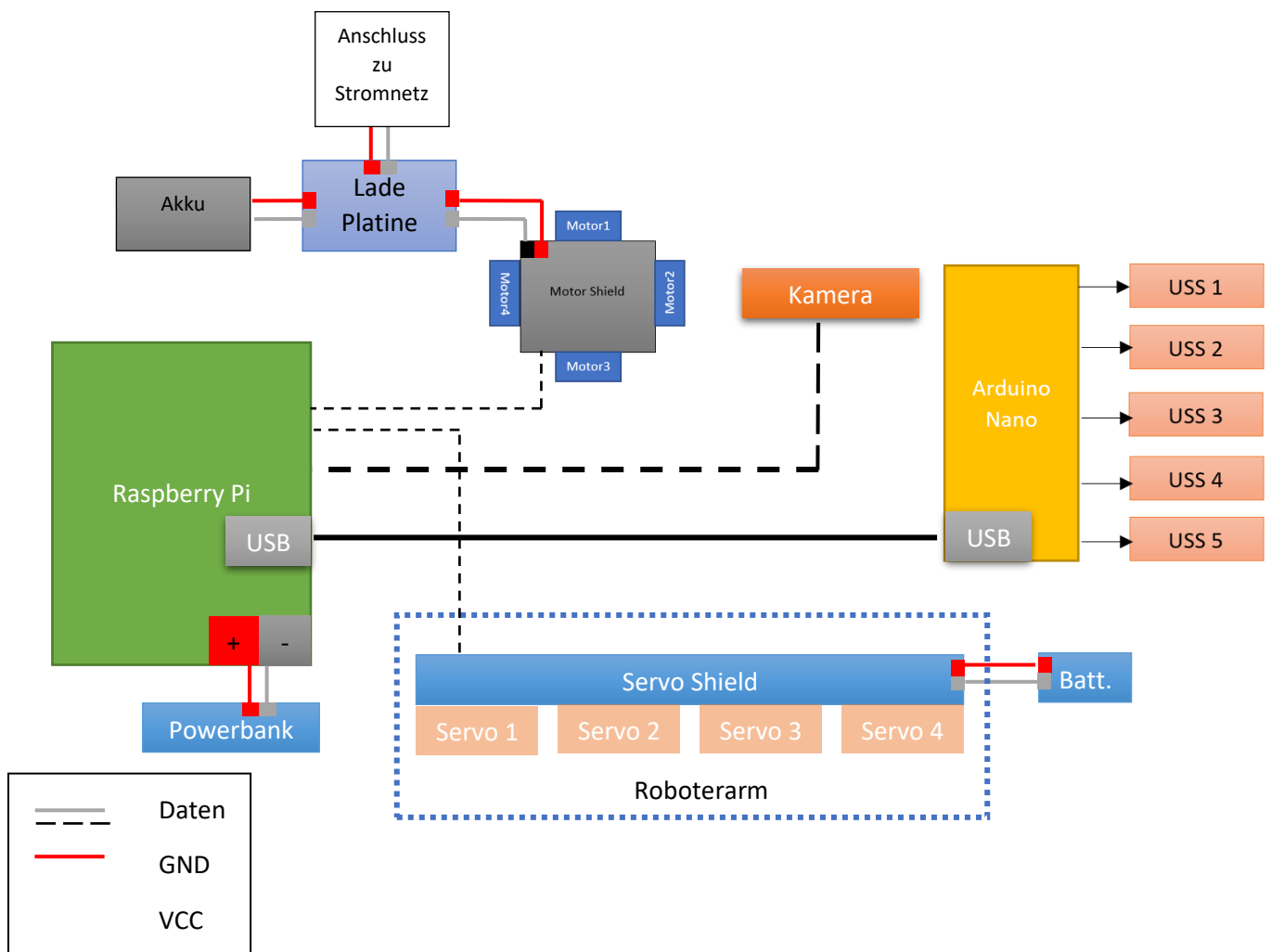
```
[ Bio_Müll;    Metllische_Dose;    Papier;    Karton_Schachtel;    Plastik    ]
```

Über die Python Bibliothek Numpy wird diese Liste als ein Array erkannt, indem der erste Listenname den Wert Null hat und das letzte den Wert Vier. Über diese Arraywerte werden anschließend die Kategoriennamen beim Erkennen eines Objektes gelesen und über die „Bounding Box“ geschrieben.

Nachdem ein Objekt erkannt wird, wird die Funktion im Roboterarmcode aufgerufen und die einzelnen Schritte zur Abfallentsorgung durchgeführt. Nach der Erkennung eines Objektes wird der Klassenname des erkannten Objektes aufgerufen und es wird ausgegeben. Danach geht der Roboterarm in seine Ausgangs- bzw. Startposition. Sobald der Roboterarm beginnt nach dem Objekt zu greifen, wird ausgegeben, dass das Objekt gegriffen wird. Zum Schluss entsorgt der Roboterarm den Müll und dies wird als Print-Befehl ausgegeben, dass der Abfall entsorgt wurde.

3.3 Modulare Umsetzung und Kommunikation im Gesamtsystem

Gezeigt ist sowohl Leistungs- als auch Datenverbindungen des Roboter-Gesamtsystems zum aktuellen Ausbaustand. Die Abbildung ist nur eine qualitative Darstellung.



3.4 Konstruktion von Fehlenden Bauteilen

Im Laufe des Projektes M4000 kam es immer wieder vor, dass gewisse Bauteile für gewisse Elemente benötigt wurden. Da der Roboter ein für sich einzigartiges Gesamtsystem ist, waren die benötigten Dinge nicht einfach zu kaufen oder zu bestellen. Aus diesem Grund wurde zur Maßnahme des 3D-Druckens gegriffen und viele Teile selbstständig mit Solid Edge konstruiert und ausgedruckt. Auch haben gewisse Teile in Bestellungen gefehlt, wodurch diese ersetzt werden mussten.

Zunächst musste eine Karosserieebene beschafft werden, da der Roboter, vor allem aufgrund der Verkabelung und der Recheneinheiten, sich auf drei Ebenen verlaufen werden sollte. Hierbei haben wir uns an der mitgelieferten Ebene des Fahrzeuggestells orientiert und diese in einem etwas größeren Maße reproduziert.

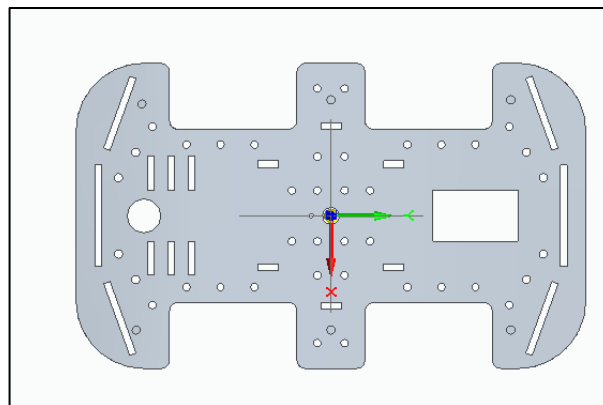


Abbildung 36: Karosserie Ebene für den Modellroboter

Insbesondere für die Sensorik und Kamera wurden Halterungen konstruiert, damit diese sicher und fest auf dem Roboter angebracht werden konnten. Hierbei war es wichtig, dass die Teile nicht viel Platz einnehmen, jedoch auch nicht zerbrechlich oder zu leicht sind. Auch war die Funktionalität eine sehr wichtige Komponente bei der Konstruktion dieser Teile. Die Funktion des Bauteils für welches ein Teil konstruiert und gedruckt wurde, musste in erster Linie nicht beeinträchtigt werden. Jedoch wurde auch auf die Optik und auf den Look der Teile geachtet, da diese indirekt den Roboter repräsentieren.

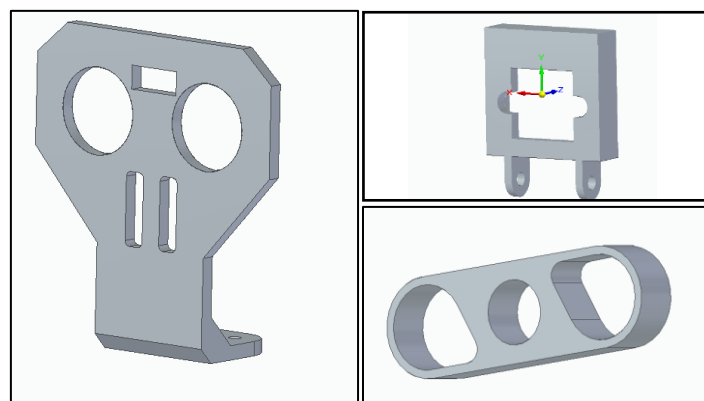


Abbildung 37: Links: USS-Halter, rechts oben: Kamerahalter, Rechtsunten: Kamerahalter mit IR-Sensoren

Für den Roboterarm musste auch ein Teil konstruiert werden, da dieses Teil in der Bestellung gefehlt hat. Hierbei musste man insbesondere darauf achten, dass das Bauteil den Kräften und Momenten des

Roboterarms und der Servomotoren standhalten kann. Wie auch oben erwähnt war die Funktionalität hier an erster Stelle. Des Weiteren musste ein Stützbein für die konstruierte Ebene angefertigt werden. Hierbei ist auch sehr gut die Individualität des Roboters zu sehen, da hier für ein selbstständig konstruiertes Teil ein weiteres selbstständig konstruiertes Teil angefertigt wurde.

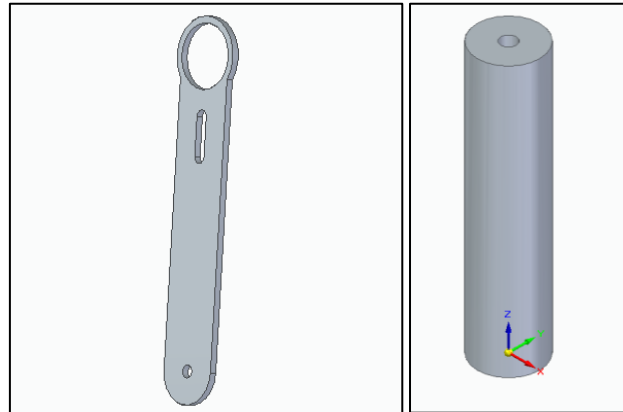


Abbildung 38: Links: Fehlende Bauteil für den Roboterarm, Rechts: Stützbein für die Konstruierte Ebene

Zu guter Letzt wurde ein Logo für den Omega-Tron konstruiert und ausgedruckt, da dieser ein Zeichen für die gemachte Arbeit und als Aushängeschild des Roboters dient.

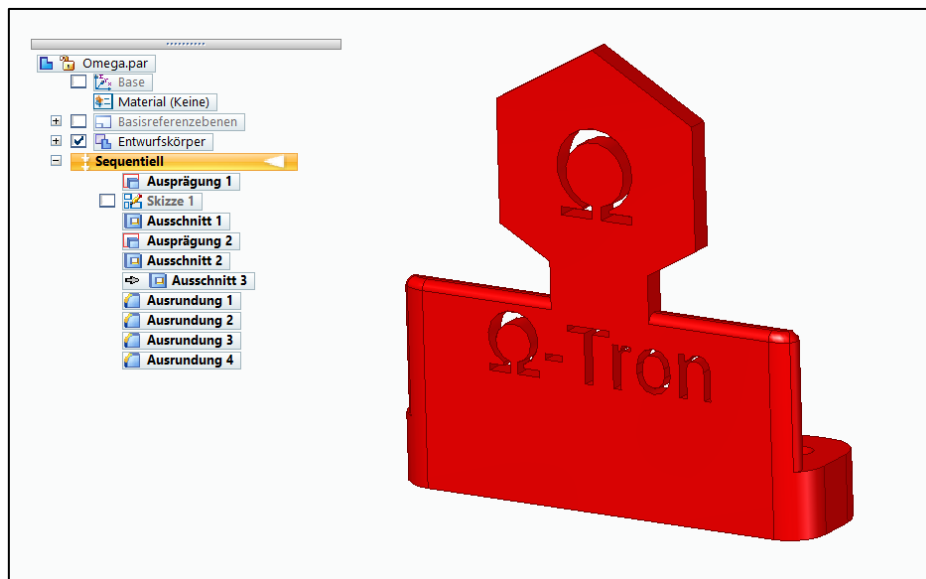


Abbildung 39: 3D-Gedruckte Logo von Omega-Tron

4 Probleme bei Umsetzung und deren Behebung

4.1 Fahrspurerkennung

4.1.1 Bildverarbeitung

Im *Kapitel 2.2.1.1 Bildverarbeitung* wurden zwei verschiedene Algorithmen zur Fahrspurerkennung vorgeschlagen. Nach mehreren Probeläufen gab es bei dem ersten Algorithmus einige Komplikationen bei der Kurvenerkennung. Die Erkennung von geraden Fahrspurlinien war hier kein Problem, jedoch war es problematisch gekrümmte Linien zu erkennen.

Das Problem wurde durch das „Birdseyeview“ Methode gelöst. Durch das Einstellen des Bildes in die Draufsicht entspricht die Form einer Fahrspur nahezu der realen Fahrbahn mit einer minimalen Verzerrung. Durch diese Transformation kann man auch gekrümmten Fahrspurlinien erkennen.

Aus diesem Grund wird dieser Algorithmus auch bei Omega-Tron implementiert und hat zuverlässige Ergebnisse ergeben. Nach einigen Probeläufen und mehreren Versuchen wurden die optimalen Werte zum Einstellen des HSV-Bildes gefunden sowohl als auch die optimale Geschwindigkeit und Sensitivität mit dem das Auto die Kurven aufnehmen kann. Dabei ist zu beachten, dass die Raspberry Pi eine Framerate von ungefähr 14 fps hat, und die Geschwindigkeit beim Lenken soll auch entsprechend eingestellt werden damit das Auto nicht schneller als die Bilder-Verarbeitungsrate ist.

4.1.2 Deep Learning Model

Beim Training des Deep Learning Models wurden mehrere Datensätze gesammelt, wobei einige von dem zu wenig Daten hatten oder eine „Overfitting“ Verhalten nach dem Trainieren gezeigt haben.

Die Ergebnisse aus der im Omega-Tron implementierten Model zeigen, dass es gut geeignet ist, um autonom durch eine Fahrbahn zu fahren. Das Modell könnte eventuell noch verbessert werden durch das Zufügen von mehr Trainingsdaten. Jedoch ist das „Overfitting“ Problem zu beachten

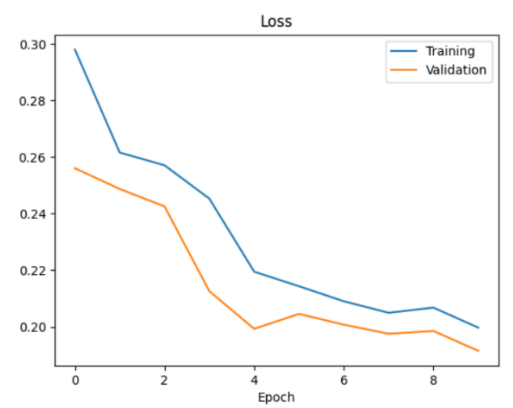


Abbildung 40: Ergebnisse des trainierten CNN-Model

Beim Implementieren des Modells gab es jedoch ein Problem beim Herunterladen der Tensorflow-Bibliothek in Raspberry Pi. Die konnte nicht richtig heruntergeladen werden da es einige Komplikationen zwischen den PIP Bibliothek, den aktuellen Python 3 Version und numpy in Raspberry gab. Das Herunterladen einer älteren Version von Tensorflow oder das Herunterladen von Tensorflow Lite hat das Problem auch nicht gelöst.

Das Installieren der Raspbian Betriebssystem oder von einem Ubuntu Version in einen anderen SD-Karte ist eine gute Lösungsansatz um mögliche defekte Bibliotheken Pakete zu aktualisieren.

4.2 Objekterkennung

Die Objekterkennung funktioniert zuverlässig bei der Erkennung von den fünf trainierten Kategorien und kann auch problemlos mit dem Roboterarm interagieren. Die Bildverarbeitung seitens Raspberry Pi ist etwas langsam, was dazu führt, dass erkannte Objekte nicht in Echtzeit dargestellt werden können. Trotz dieses Problems ist dies kein Hindernis für das zuverlässige Erkennen und Greifen von Objekten.

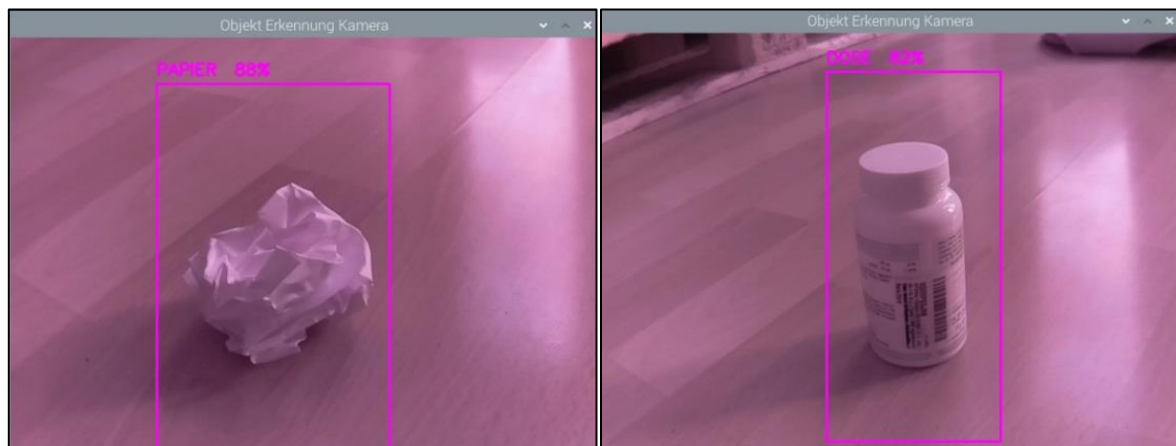


Abbildung 41: Objekterkennung mit Yolo

4.3 Roboterarm

Beim Roboterarm gab es von Anfang an insbesondere mechanische und Hardwaretechnische Probleme.

Nachdem das Paket mit den Teilen für den Roboterarm angekommen ist, ist sofort aufgefallen, dass gewisse Bauteile, welche von immenser Bedeutung für die Funktionalität des Greifers sind, fehlen, sodass diese zunächst einmal nachkonstruiert und anschließend 3D gedruckt werden mussten. Trotz sorgfältigem und sauberem Arbeiten ist das Ersatzteil auf Grund des Materials des 3D Druckers nicht auf das gleiche qualitative Niveau des Originalteils zu stellen.

Ein weiteres Problem war, dass der Servotreiber der bestellt wurde, keine Verbindung zum Raspberry Pi herstellen konnte, wodurch ein Ersatzteil beschafft werden musste. Hierbei wurde man glücklicherweise Elektro-Labor des C-Labs fündig und konnte dieses Problem, ohne zu sehr Zeit zu verlieren beseitigen.

Das größte Problem beim Roboterarm war die Spannungsversorgung. Hierbei war vorausgesetzt, dass die Servomotoren eine Spannung von 5V und einen Strom von 1 A benötigen. Die Batterien, an denen der Roboterarm angeschlossen war, haben diese Versorgung ohne Probleme zunächst aufgewiesen. Jedoch hat man nach einer sehr kurzen Zeit schon gesehen, dass die Servomotoren nicht mehr die Leistung aufbringen können, welche notwendig war, um die Gelenke zu bewegen. Als die Batterien gemessen wurden, hat man einen starken Spannungsabfall messen können. Auch beim zweiten Versuch und mit frischen Batterien ist dieses Problem nicht beseitigt werden können. Zu Testzwecken wurde ein vom Lieferanten mitgeliefertes Netzteil verwendet. Schließlich wurde dazu entschieden den Roboterarm mit einer Powerbank zu betreiben, da hier der Spannungsabfall nicht bemerkbar war.

4.4 Hindernisumfahrung

Die Ultraschallsensoren funktionieren problemlos, wenn die einzeln betrieben werden. Bei der Verwendung aller Sensoren, werden die Trigger von einzelnen Sensoren von allen Sensoren gemessen. Dies führt zu einer Fehlinterpretation der Entfernungen in der jeweiligen Richtung. Dadurch kommt es zu Fehlerhafte Entfernungsmessung.

Durch ein Vereinzeln der Messungen kann das Problem umgangen werden. Jedoch führt das zu einen geringeren Abtastrate der Entfernungsmessung.

4.5 Motorsteuerung

Bei der Verwendung von AMSpi ist beim schreiben des Motortreibers das Problem entstanden, dass ein GPIO cleanup nach den Funktionsaufrufen stattfand, dadurch wurden die Motoren faktisch nicht angesteuert. Eine andre Programmstruktur löste diesen Fehler. Bei der Handsteuerung fiel das Problem auf, dass der Roboter ohne Ansteuerung den letzten Fahrbefehl beibehielt. Dies wurde durch einen nach einem Timer gesteuerten Stopp Befehl gelöst.

4.6 Ortung und Navigation

Bei der Installation von ROS/ dessen Module stellte sich das Betriebssystem Raspbian leider als nicht überwindbares Hindernis heraus. Für eine erfolgreiche Implementierung muss der Roboter am besten auf Ubuntu Mate umgestellt werden. Dafür benötigt man dort einen Workaround zum Verwenden der GPIO' s.

4.7 Induktives Laden

Beim Bestellen wurde die Station vergessen. Deswegen und auf Grund der mangelnden Navigationsfähigkeit, haben wir uns dazu entschlossen, die Zielsetzung hintenanzustellen. Bis auf die Ladestation ist die IMU und die Ladespule aber physisch vorhanden nur aufgrund des fehlenden Nutzens noch nicht angeschlossen. Die Ladezustandsmessung benötigt noch eine Verstärkerschaltung um die 200mV auf die zum Messen benötigten 3V zu verstärken.

5 Zusammenfassung und Ausblick

Alles in allem verfügt Omega-Tron über ein hohes Maß an Zuverlässigkeit bei der Durchführung der Aufgaben für die einzelnen Module, die Motorsteuerung, die Steuerung des Roboterarms, Fahrspurerkennung und Objekterkennung.

Außerdem besteht ein hohes Potential an Verbesserungen der jeweiligen Module durch Erweiterung und Weiterentwicklung. Beispielsweise das Einbauen einer TPU (Tensor Processing Unit) würde den Prozess beim autonomen Fahren und bei der Objekterkennung deutlich verbessern, womit der Raspberry Pi die betreffenden Module schneller ausführen würde.

Aktuell erkennt der Roboter den vor ihm liegenden Abfall, wodurch der Greifarm-Befehl automatisch zum Einsatz kommt. Eine Verbesserung, die auch bereits in der Praxis Anwendung findet, ist, dass der Roboter entlang des fahrenden Weges Abfall von weiterer Entfernung erkennt und gezielt dorthin fährt. Dies ist durch das Hinzufügen von Modulen im Hauptskript und das Aufrufen vom Motor- und USS-Modul realisierbar. Durch das Trainieren von größeren Datensätzen, kann die Klassifizierungsgenauigkeit erhöht werden.

Bei der Wahrnehmung der Umgebung, um eine Datenkollision zu verhindern, welches durch Interferenz der USS zur Stande kommt, wäre es besser die einzelnen Entfernungsmessung mit zeitlich größerem Abstand zu legen.

Während dem Projekt sind die Komplexität und der Umfang eines solchen Modellroboterprojektes jedem von uns sehr deutlich aufgezeigt worden, was Motivation und Anreiz schaffte, um die inneren Prozesse bei autonom fahrendem Roboter zu erlernen und die theoretischen Grundlagen in diesem Bereich zu vertiefen.

Zum Abschluss möchten wir uns an das gesamte C-Lab für die Werkzeug-, als auch Materialbereitstellung bedanken. Außerdem war durch die Bereitstellung Ihrer Räumlichkeiten stets ein produktives Arbeiten möglich.

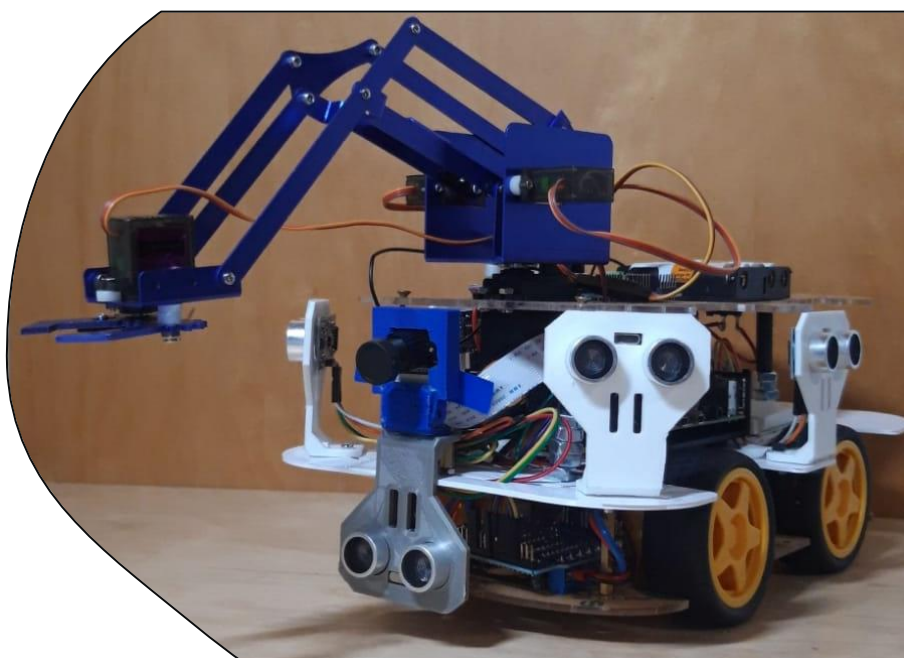


Abbildung 42: Omega-Tron

6 Anhang

6.1 Anforderungsliste

6.2 Projektplan

6.3 Bestellliste

6.4 Programcode

6.4.1 Motorsteuerung

6.4.2 Hindernisumfahrung

6.4.3 Fahrspurerkennung

6.4.4 Objekterkennung

6.4.5 Roboterarm

Abbildungsverzeichnis

Abbildung 1: Sensortopologie für Autonome Fahrzeuge.....	3
Abbildung 2: Edag Citybot ©Edag.....	4
Abbildung 3: Amazon Scout Postbote ©Amazon.....	4
Abbildung 4: Fahrspurverfolgung durch Kantenerkennung mit Hough Transformation.....	7
Abbildung 5: Fahrspurerkennung durch Farbenerkennung.....	7
Abbildung 6: CNN Schichten.....	8
Abbildung 7: Verbindung der Raspberry Pi mit 5 USS.....	9
Abbildung 8: verbindung der Raspberry Pi mit Raspberry Pi Pico (Hier Arduino Nano).....	10
Abbildung 9: Objekt- und Positionserkennung von Fahrzeugen im Straßenverkehr.....	10
Abbildung 10: Links Getriebe Motor, Rechts Motorshield.....	11
Abbildung 11: Beispielbild TIAGo Map.....	12
Abbildung 12: Schaltschema Servomotoren.....	13
Abbildung 13: Roboterarm.....	13
Abbildung 14: Ladeplatine mit Verkabelung Ladezustandsmessung.....	14
Abbildung 15: Raspberry Pi 4 Kommunikation mit USS.....	14
Abbildung 16: Raspberry Pi I2C mit Raspberry Pico.....	15
Abbildung 17: Raspberry PI Serielle Kommunikation mit Arduino Nano.....	15
Abbildung 18: Darstellung der Fahrspur in HSV-Farben Spektrum.....	17
Abbildung 19: Schieberegler Fenster zum Einstellen der HSV Werte.....	17
Abbildung 20: Schematische Darstellung der Bildtransformation in der Draufsicht.....	17
Abbildung 21: Perspektive Transformation.....	18
Abbildung 22: Koordinaten der Aktuellen Bild (Links) und mit Transformierte Koordinaten in HSV-Farben (Rechts).....	18
Abbildung 23: Aufsummierung der Pixeln zur Kurven Verfolgung.....	19
Abbildung 24: Auszug aus der Implementierung der Algorithmus.....	20
Abbildung 25: Schema der Datensammlung Programm Code.....	21
Abbildung 26: Auszug aus der Daten Import.....	22
Abbildung 27: Visualisierung der Daten.....	22
Abbildung 28: Ausgegliche Daten.....	23
Abbildung 29: Auszug aus der Programcode.....	23
Abbildung 30: NVIDIA CNN Model.....	24
Abbildung 31: Trainieren des Models.....	26
Abbildung 32: Ergebnisse aus der Training des Models.....	26
Abbildung 33: Öffentliches Bild (links) und selbst geschossenes Bild (rechts).....	27
Abbildung 34: Dose mit Bounding Box.....	27
Abbildung 35: x = Anzahl Zyklen, y = Prozentualer Fehler.....	28
Abbildung 36: Karosserie Ebene für den Modellroboter.....	30
Abbildung 37: Links: USS-Halter, rechts oben: Kamerahalter, Rechtsunten: Kamerahalter mit IR-Sensoren.....	30
Abbildung 38: Links: Fehlende Bauteil für den Roboterarm, Rechts: Stützbeine für die Konstruierte Ebene.....	31
Abbildung 39: 3D-Gedruckte Logo von Omega-Tron.....	31
Abbildung 40: Ergebnisse des trainierten CNN-Model.....	32
Abbildung 41: Objekterkennung mit Yolo.....	33
Abbildung 42: Omega-Tron.....	35

Literaturverzeichnis

Fahrspurerkennung

https://easychair.org/publications/preprint_download/4DRI

<https://towardsdatascience.com/a-deep-dive-into-lane-detection-with-hough-transform-8f90fdd1322f>

<https://towardsdatascience.com/deeppicar-part-1-102e03c83f2c>

<https://towardsdatascience.com/deeppicar-part-4-lane-following-via-opencv-737dd9e47c96>

<https://github.com/TurtleTaco/Advanced-Lane-Detection>

http://www.gm.fh-koeln.de/~hk/lehre/ala/ws0506/Praktikum/Projekt/E_rot/Dokumentation.pdf

<https://www.hindawi.com/journals/js/2016/4058093/>

https://sbme-tutorials.github.io/2019/cv/notes/4_week4.html

<https://github.com/naokishibuya/car-finding-lane-lines>

https://www.researchgate.net/figure/Lane-detection-with-Hough-transform-performed-using-OpenCV_fig3_332897463

http://coecsl.ece.illinois.edu/ge423/spring05/group8/FinalProject/HSV_writeup.pdf

<https://www.opencv-srf.com/2010/09/object-detection-using-color-seperation.html>

<https://stackoverflow.com/questions/22588146/tracking-white-color-using-python-opencv>

https://en.wikipedia.org/wiki/HSL_and_HSV

<https://www.computervision.zone/courses/self-driving-car-using-raspberry-pi/>

https://wiki.hshl.de/wiki/index.php/Kamerabildtransformation_in_Vogelperspektive

<https://projects-raspberry.com/curved-lane-detection/>

https://github.com/yukitsuji/Advanced_Lane_Finding

<https://www.youtube.com/watch?v=PZsbROEpFNU>

<https://www.hindawi.com/journals/js/2016/4058093/>

<https://stackoverflow.com/questions/43226702/image-warping-using-opencv>

<https://medium.com/analytics-vidhya/opencv-perspective-transformation-9edffeb2143>

<https://theailearner.com/tag/cv2-getperspectivetransform/>

<https://www.geeksforgeeks.org/perspective-transformation-python-opencv/>

<https://www.programcreek.com/python/example/85017/cv2.getPerspectiveTransform>

<https://manivannan-ai.medium.com/set-trackbar-on-image-using-opencv-python-58c57fbee1ee>

<https://stackoverflow.com/questions/39362497/how-can-i-set-the-default-position-of-a-trackbar-in-opencv>

<https://www.programcreek.com/python/example/70426/cv2.createTrackbar>

<https://www.programcreek.com/python/example/70423/cv2.getTrackbarPos>

Deep Learning und Convolutional Neural Networks

<https://news.microsoft.com/de-de/microsoft-erklaert-was-ist-deep-learning-definition-funktionen-von-dl/>

<https://www.kaggle.com/ryanholbrook/a-single-neuron>

<https://towardsdatascience.com/deepcar-part-5-lane-following-via-deep-learning-d93acdce6110>

<https://github.com/PacktPublishing/Applied-Deep-Learning-and-Computer-Vision-for-Self-Driving-Cars>

<https://towardsdatascience.com/deepcar-part-5-lane-following-via-deep-learning-d93acdce6110>

<https://developer.nvidia.com/drive/drive-perception>

Nvidia model

<https://developer.nvidia.com/blog/deep-learning-self-driving-cars/>

<https://images.nvidia.com/content/tegra/automotive/images/2016/solutions/pdf/end-to-end-dl-using-px.pdf>

<https://github.com/tbchhetri/selfDrivingCar>

<https://github.com/shawpan/drive>

Objekterkennung

https://www.matse.itc.rwth-aachen.de/dienste/public/show_document.php?id=1875

<https://www.youtube.com/watch?v=F5KpG7fcYaY>

<https://jonathan-hui.medium.com/real-time-object-detection-with-yolo-yolov2-28b1b93e2088>

Abb.9: <https://blog.330ohms.com/2020/11/17/deteccion-de-objetos-con-yolo/>

Abb21: <https://www.flickr.com/photos/30478819@N08/50635298728>

Abkürzungsverzeichnis

ACC	Adaptive Cruise Control
CNN	Convolutional Neural Network
Eng.	Englisch (Hier englische Fachbegriff)
HSV	Hue/Saturation/Value Farbenspektrum
LKAS	Lane Keep Assistance System
RGB	Red/Blue/Green Farbenspektrum
USS	Ultraschall Sensor